

codex-windows / 019f28c1-5989-7a92-a03b-29cd0860a536

Metadata

```
- Source: `codex-windows`  
- Kind: `codex`  
- Source file: `C:\Users\avidu\codex\archived_sessions\rollout-2026-07-03T21-43-07-019f28c1-5989-7a92-a03b-29cd0860a536.jsonl`  
- SHA-256: `4017072b6295ef915810b823f1367d885fa9617a3967c82a5d93244b1f0c8b8d`  
- Source modified: `2026-07-04T01:29:10+00:00`  
- Imported at: `2026-07-05T16:48:16+00:00`  
- cli_version: `0.142.5`  
- cwd: `C:\Users\avidu\Projects\khelsutra-guru`  
- model_provider: `openai`  
- session_id: `019f28c1-5989-7a92-a03b-29cd0860a536`  
- source: `vscode`
```

Transcript

1. developer (2026-07-03T16:13:10.148Z)

```
<permissions instructions>  
Filesystem sandboxing defines which files can be read or written. `sandbox_mode` is `danger-  
full-access`: No filesystem sandboxing - all commands are permitted. Network access is enabled.  
Approval policy is currently never. Do not provide the `sandbox_permissions` for any reason,  
commands will be rejected.  
</permissions instructions>  
<app-context>
```

Codex desktop context

- You are running inside the Codex (desktop) app, which allows some additional features not available in the CLI alone:

Images/Visuals/Files

- In the app, the model can display images and videos using standard Markdown image syntax: `![alt](url)`
- When sending or referencing a local image or video, always use an absolute filesystem path in the Markdown image tag (e.g., `![alt](/absolute/path.png)`); relative paths and plain text will not render the media.
- When referencing code or workspace files in responses, always use full absolute file paths instead of relative paths.
- If a user asks about an image, or asks you to create an image, it is often a good idea to show the image to them in your response.
- Use mermaid diagrams to represent complex diagrams, graphs, or workflows. Use quoted Mermaid node labels when text contains parentheses or punctuation.
- Return web URLs as Markdown links (e.g., `[label](https://example.com)`).

Workspace Dependencies

- For sheets, slides, and documents, call ``load_workspace_dependencies`` to find the bundled runtime and libraries.

Automations

- This app supports recurring automations, reminders, monitors, follow-ups, and thread wakeups. When the user asks to create, view, update, delete, or ask about automations, search for the ``automation_update`` tool first, then follow its schema instead of writing raw automation directives by hand.

- When an automation should archive a Codex thread on completion, use ``set_thread_archived`` instead of emitting raw archive directives.

Thread Coordination

- When the user asks to create, fork, inspect, continue, hand off, pin, archive, rename, or otherwise manage Codex threads, search for the relevant thread tool first: ``create_thread``, ``fork_thread``, ``list_threads``, ``read_thread``, ``send_message_to_thread``, ``handoff_thread``,

`set\_thread\_pinned`, `set\_thread\_archived`, or `set\_thread\_title`.

- Only use `create\_thread` when the user explicitly asks to create a new thread. Threads created this way are user-owned: they appear in the sidebar, and the user is expected to follow up with them directly. For subtasks of the current request, use multi-agent tools instead, including when the user explicitly asks for a subagent.
- After a successful `create\_thread` call, emit `::created-thread{threadId="..."}` for a created thread or `::created-thread{pendingWorktreeId="..."}` for queued worktree setup on its own line in your final response.

Inline Code Comments

- Use the `::code-comment{...}` directive when you need to attach feedback directly to specific code lines.
- Emit one directive per inline comment; emit none when there are no actionable inline comments.
- Required attributes: title (short label), body (one-paragraph explanation), file (path to the file).
- Optional attributes: start, end (1-based line numbers), priority (0-3).
- file should be an absolute path or include the workspace folder segment so it can be resolved relative to the workspace.
- Keep line ranges tight; end defaults to start.
- Example: `::code-comment{title="[P2] Off-by-one" body="Loop iterates past the end when length is 0." file="/path/to/foo.ts" start=10 end=11 priority=2}`

```
</app-context>
<collaboration_mode># Collaboration Mode: Default
```

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different ``<collaboration_mode>...</collaboration_mode>`` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

request_user_input availability

Use the `request\_user\_input` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user's request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.

```
</collaboration_mode>
<apps_instructions>
```

Apps (Connectors)

Apps (Connectors) can be explicitly triggered in user messages in the format `[\${app-name}](app://{connector\_id})`. Apps can also be implicitly triggered as long as the context suggests usage of available apps.

An app is equivalent to a set of MCP tools within the `codex\_apps` MCP.

An installed app's MCP tools are either provided to you already, or can be lazy-loaded through the `tool\_search` tool. If `tool\_search` is available, the apps that are searchable by `tools\_search` will be listed by it.

Do not additionally call `list\_mcp\_resources` or `list\_mcp\_resource\_templates` for apps.

```
</apps_instructions>
<skills_instructions>
```

Skills

A skill is a set of local instructions to follow that is stored in a `SKILL.md` file. Below is the list of skills that can be used. Each entry includes a name, description, and a short path that can be expanded into an absolute path using the skill roots table.

Skill roots

- `r0` = `C:/Users/avidu/.agents/skills`
- `r1` = `C:/Users/avidu/.codex/skills/.system`
- `r2` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/anthropic-skills/1.0.0/skills`
- `r3` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/cowork-plugin-management/0.2.2/skills`

- `r4` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/design/1.2.0/skills`
- `r5` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/engineering/1.2.0/skills`
- `r6` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/productivity/1.3.0/skills`
- `r7` = `C:/Users/avidu/.codex/plugins/cache/openai-bundled`
- `r8` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/github/0.1.5/skills`
- `r9` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/gmail/0.1.3/skills`
- `r10` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-calendar/1.2.3/skills`
- `r11` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-drive/0.1.7/skills`
- `r12` = `C:/Users/avidu/.codex/plugins/cache/openai-primary-runtime`

Available skills

- imagegen: Generate or edit raster images when the task benefits from AI-created bitmap visuals such as photos, illustrations, textures, sprites, mockups, or transparent-background cutouts. Use when Codex should create a brand-new image, transform an existing image, or derive visual variants from references, and the output should be a bitmap asset rather than repo-native code or vector. Do not use when the task is better handled by editing e (file: r1/imagegen/SKILL.md)
- openai-docs: Use when the user asks how to build with OpenAI products or APIs, asks about Codex itself or choosing Codex surfaces, needs up-to-date official documentation with citations, help choosing the latest model for a use case, or model upgrade and prompt-upgrade guidance; use OpenAI docs MCP tools for non-Codex docs questions, use the Codex manual helper first for broad Codex self-knowledge, and restrict fallback browsing to official (file: r1/openai-docs/SKILL.md)
- plugin-creator: Create and scaffold plugin directories for Codex with a required `.codex-plugin/plugin.json`, optional plugin folders/files, valid manifest defaults, and personal-marketplace entries by default. Use when Codex needs to create a new personal plugin, add optional plugin structure, generate or update marketplace entries for plugin ordering and availability metadata, or update an existing local plugin during development with the C (file: r1/plugin-creator/SKILL.md)
- skill-creator: Guide for creating effective skills. This skill should be used when users want to create a new skill (or update an existing skill) that extends Codex's capabilities with specialized knowledge, workflows, or tool integrations. (file: r1/skill-creator/SKILL.md)
- skill-installer: Install Codex skills into \$CODEX_HOME/skills from a curated list or a GitHub repo path. Use when a user asks to list installable skills, install a curated skill, or install a skill from another repo (including private repos). (file: r1/skill-installer/SKILL.md)
- anthropic-skills:consolidate-memory: Reflective pass over your memory files ? merge duplicates, fix stale facts, prune the index. (file: r2/consolidate-memory/SKILL.md)
- anthropic-skills:doc-coauthoring: Guide users through a structured workflow for co-authoring documentation. Use when user wants to write documentation, proposals, technical specs, decision docs, or similar structured content. This workflow helps users efficiently transfer context, refine content through iteration, and verify the doc works for readers. Trigger when user mentions writing docs, creating proposals, drafting specs, or similar documentation tasks. (file: r2/doc-coauthoring/SKILL.md)
- anthropic-skills:docx: Use this skill whenever the user wants to create, read, edit, or manipulate Word documents (.docx files). Triggers include: any mention of 'Word doc', 'word document', '.docx', or requests to produce professional documents with formatting like tables of contents, headings, page numbers, or letterheads. Also use when extracting or reorganizing content from .docx files, inserting or replacing images in documents, performing find-an (file: r2/docx/SKILL.md)
- anthropic-skills:new-session-hello: I want claude to read the github repos linked and have some context when I start a new coding session (file: r2/new-session-hello/SKILL.md)
- anthropic-skills:pdf: Use this skill whenever the user wants to do anything with PDF files. This includes reading or extracting text/tables from PDFs, combining or merging multiple PDFs into one, splitting PDFs apart, rotating pages, adding watermarks, creating new PDFs, filling PDF forms, encrypting/decrypting PDFs, extracting images, and OCR on scanned PDFs to make them searchable. If the user mentions a .pdf file or asks to produce one, use this (file: r2/pdf/SKILL.md)
- anthropic-skills:pptx: Use this skill any time a .pptx file is involved in any way ? as input, output, or both. This includes: creating slide decks, pitch decks, or presentations; reading, parsing, or extracting text from any .pptx file (even if the extracted content will be used elsewhere, like in an email or summary); editing, modifying, or updating existing presentations; combining or splitting slide files; working with templates, layouts, speaker (file: r2/pptx/SKILL.md)
- anthropic-skills:schedule: Create or update a scheduled task that runs automatically. Use when the user says things like "every day", "each morning", "remind me in an hour", "run this at noon", or wants to reschedule an existing task. (file: r2/schedule/SKILL.md)

- anthropic-skills:setup-cowork: Guided Cowork setup ? install role-matched plugins, connect your tools, try a skill. (file: r2/setup-cowork/SKILL.md)
- anthropic-skills:skill-creator: Create new skills, modify and improve existing skills, and measure skill performance. Use when users want to create a skill from scratch, edit, or optimize an existing skill, run evals to test a skill, benchmark skill performance with variance analysis, or optimize a skill's description for better triggering accuracy. (file: r2/skill-creator/SKILL.md)
- anthropic-skills:xlsx: Use this skill any time a spreadsheet file is the primary input or output. This means any task where the user wants to: open, read, edit, or fix an existing .xlsx, .xlsm, .csv, or .tsv file (e.g., adding columns, computing formulas, formatting, charting, cleaning messy data); create a new spreadsheet from scratch or from other data sources; or convert between tabular file formats. Trigger especially when the user references a spr (file: r2/xlsx/SKILL.md)
- browser:control-in-app-browser: Control the in-app Browser. Use to open, navigate, inspect, test, click, type, screenshot, or verify local targets such as localhost, 127.0.0.1, ::1, file://, the current in-app browser tab, and websites shown side by side inside Codex. (file: r7/browser/26.623.101652/skills/control-in-app-browser/SKILL.md)
- chrome:control-chrome: Control the user's Chrome browser for tasks that depend on existing Chrome state: tabs, logged-in sessions, or extensions. Prefer purpose-built connectors, APIs, or CLIs when available. (file: r7/chrome/26.623.101652/skills/control-chrome/SKILL.md)
- computer-use:computer-use: Control Windows apps from Codex (file: r7/computer-use/26.623.101652/skills/computer-use/SKILL.md)
- cowork-plugin-management:cowork-plugin-customizer: Customize a Claude Code plugin for a specific organization's tools and workflows. Use when: customize plugin, set up plugin, configure plugin, tailor plugin, adjust plugin settings, customize plugin connectors, customize plugin skill, tweak plugin, modify plugin configuration. (file: r3/cowork-plugin-customizer/SKILL.md)
- cowork-plugin-management:create-cowork-plugin: Guide users through creating a new plugin from scratch in a cowork session. Use when users want to create a plugin, build a plugin, make a new plugin, develop a plugin, scaffold a plugin, start a plugin from scratch, or design a plugin. This skill requires Cowork mode with access to the outputs directory for delivering the final .plugin file. (file: r3/create-cowork-plugin/SKILL.md)
- design:accessibility-review: Run a WCAG 2.1 AA accessibility audit on a design or page. Trigger with "audit accessibility", "check a11y", "is this accessible?", or when reviewing a design for color contrast, keyboard navigation, touch target size, or screen reader behavior before handoff. (file: r4/accessibility-review/SKILL.md)
- design:design-critique: Get structured design feedback on usability, hierarchy, and consistency. Trigger with "review this design", "critique this mockup", "what do you think of this screen?", or when sharing a Figma link or screenshot for feedback at any stage from exploration to final polish. (file: r4/design-critique/SKILL.md)
- design:design-handoff: Generate developer handoff specs from a design. Use when a design is ready for engineering and needs a spec sheet covering layout, design tokens, component props, interaction states, responsive breakpoints, edge cases, and animation details. (file: r4/design-handoff/SKILL.md)
- design:design-system: Audit, document, or extend your design system. Use when checking for naming inconsistencies or hardcoded values across components, writing documentation for a component's variants, states, and accessibility notes, or designing a new pattern that fits the existing system. (file: r4/design-system/SKILL.md)
- design:research-synthesis: Synthesize user research into themes, insights, and recommendations. Use when you have interview transcripts, survey results, usability test notes, support tickets, or NPS responses that need to be distilled into patterns, user segments, and prioritized next steps. (file: r4/research-synthesis/SKILL.md)
- design:user-research: Plan, conduct, and synthesize user research. Trigger with "user research plan", "interview guide", "usability test", "survey design", "research questions", or when the user needs help with any aspect of understanding their users through research. (file: r4/user-research/SKILL.md)
- design:ux-copy: Write or review UX copy ? microcopy, error messages, empty states, CTAs. Trigger with "write copy for", "what should this button say?", "review this error message", or when naming a CTA, wording a confirmation dialog, filling an empty state, or writing onboarding text. (file: r4/ux-copy/SKILL.md)
- documents:documents: Create, edit, redline, and comment on .docx, Word, and Google Docs-targeted document artifacts inside the container, with a strict render-and-verify workflow. Use `render\_docx.py` to generate page PNGs (and optional PDF) for visual QA, then iterate until layout is flawless before delivering the final document. (file: r12/documents/26.630.12135/skills/documents/SKILL.md)

- engineering:architecture: Create or evaluate an architecture decision record (ADR). Use when choosing between technologies (e.g., Kafka vs SQS), documenting a design decision with trade-offs and consequences, reviewing a system design proposal, or designing a new component from requirements and constraints. (file: r5/architecture/SKILL.md)
- engineering:code-review: Review code changes for security, performance, and correctness. Trigger with a PR URL or diff, "review this before I merge", "is this code safe?", or when checking a change for N+1 queries, injection risks, missing edge cases, or error handling gaps. (file: r5/code-review/SKILL.md)
- engineering:debug: Structured debugging session ? reproduce, isolate, diagnose, and fix. Trigger with an error message or stack trace, "this works in staging but not prod", "something broke after the deploy", or when behavior diverges from expected and the cause isn't obvious. (file: r5/debug/SKILL.md)
- engineering:deploy-checklist: Pre-deployment verification checklist. Use when about to ship a release, deploying a change with database migrations or feature flags, verifying CI status and approvals before going to production, or documenting rollback triggers ahead of time. (file: r5/deploy-checklist/SKILL.md)
- engineering:documentation: Write and maintain technical documentation. Trigger with "write docs for", "document this", "create a README", "write a runbook", "onboarding guide", or when the user needs help with any form of technical writing ? API docs, architecture docs, or operational runbooks. (file: r5/documentation/SKILL.md)
- engineering:incident-response: Run an incident response workflow ? triage, communicate, and write postmortem. Trigger with "we have an incident", "production is down", an alert that needs severity assessment, a status update mid-incident, or when writing a blameless postmortem after resolution. (file: r5/incident-response/SKILL.md)
- engineering:standup: Generate a standup update from recent activity. Use when preparing for daily standup, summarizing yesterday's commits and PRs and ticket moves, formatting work into yesterday/today/blockers, or structuring a few rough notes into a shareable update. (file: r5/standup/SKILL.md)
- engineering:system-design: Design systems, services, and architectures. Trigger with "design a system for", "how should we architect", "system design for", "what's the right architecture for", or when the user needs help with API design, data modeling, or service boundaries. (file: r5/system-design/SKILL.md)
- engineering:tech-debt: Identify, categorize, and prioritize technical debt. Trigger with "tech debt", "technical debt audit", "what should we refactor", "code health", or when the user asks about code quality, refactoring priorities, or maintenance backlog. (file: r5/tech-debt/SKILL.md)
- engineering:testing-strategy: Design test strategies and test plans. Trigger with "how should we test", "test strategy for", "write tests for", "test plan", "what tests do we need", or when the user needs help with testing approaches, coverage, or test architecture. (file: r5/testing-strategy/SKILL.md)
- github:gh-address-comments: Address actionable GitHub pull request review feedback. Use when the user wants to inspect unresolved review threads, requested changes, or inline review comments on a PR, then implement selected fixes. Use the GitHub app for PR metadata and flat comment reads, and use the bundled GraphQL script via `gh` whenever thread-level state, resolution status, or inline review context matters. (file: r8/gh-address-comments/SKILL.md)
- github:gh-fix-ci: Use when a user asks to debug or fix failing GitHub PR checks that run in GitHub Actions. Use the GitHub app from this plugin for PR metadata and patch context, and use `gh` for Actions check and log inspection before implementing any approved fix. (file: r8/gh-fix-ci/SKILL.md)
- github:github: Triage and orient GitHub repository, pull request, and issue work through the connected GitHub app. Use when the user asks for general GitHub help, wants PR or issue summaries, or needs repository context before choosing a more specific GitHub workflow. (file: r8/github/SKILL.md)
- github:yeet: Publish local changes to GitHub by confirming scope, committing intentionally, pushing the branch, and opening a draft PR through the GitHub app from this plugin, with `gh` used only as a fallback where connector coverage is insufficient. (file: r8/yeet/SKILL.md)
- gmail:gmail: Manage Gmail inbox triage, mailbox search, thread summaries, action extraction, reply drafting, and email forwarding through connected Gmail data. Use when the user wants to inspect a mailbox or thread, search email with Gmail query syntax, summarize messages, extract decisions and follow-ups, prepare replies or forwarded messages, or organize messages with explicit confirmation before send, archive, delete, or label actions. (file: r9/gmail/SKILL.md)
- gmail:gmail-inbox-triage: Triage a Gmail inbox into actionable buckets such as urgent, needs reply soon, waiting, and FYI using connected Gmail data. Use when the user asks to triage the inbox, rank what needs attention, find what still needs a reply, or separate important mail from noise. (file: r9/gmail-inbox-triage/SKILL.md)

- google-calendar:google-calendar: Manage scheduling and conflicts in connected Google Calendar data. Use when the user wants to inspect calendars, compare availability, review conflicts, find a meeting room, review event notes or attachments, add or adjust reminders, place temporary holds, or draft exact create, update, reschedule, or cancel changes with timezone-aware details. (file: r10/google-calendar/SKILL.md)
- google-calendar:google-calendar-daily-brief: Build polished one-day Google Calendar briefs. Use when the user asks for today, tomorrow, or a specific date summary with an agenda, conflict flags, free windows, remaining-meeting readouts, or a calendar brief, and the Google Calendar connector is available. (file: r10/google-calendar-daily-brief/SKILL.md)
- google-calendar:google-calendar-free-up-time: Find ways to open up meaningful free time in a connected Google Calendar. Use when the user wants to clear up their day, make room for focus time, create a longer uninterrupted block, or see the smallest set of calendar changes that would give time back. (file: r10/google-calendar-free-up-time/SKILL.md)
- google-calendar:google-calendar-group-scheduler: Find and rank good meeting times for multiple people using connected Google Calendar data. Use when the user wants to schedule a group meeting, compare candidate slots across several attendees, find the best compromise time, or add a room check after narrowing the attendee-compatible options. (file: r10/google-calendar-group-scheduler/SKILL.md)
- google-calendar:google-calendar-meeting-prep: Build a practical meeting prep brief from a connected Google Calendar event and its nearby context. Use when the user wants to prepare for an upcoming meeting, understand what to read beforehand, pull in linked notes or docs, or get a concise brief on what the meeting appears to require. (file: r10/google-calendar-meeting-prep/SKILL.md)
- google-drive:google-docs: Connector-first Google Docs creation and editing in local Codex plugin sessions, with direct native create and batchUpdate workflows for simple docs, DOCX-first import for polished deliverables, target-document checks, smart chip and building-block reconstruction, connector-readback verification, and reference routing for formatting, citations, tables, and write-safety. (file: r11/google-docs/SKILL.md)
- google-drive:google-drive: Use connected Google Drive as the single entrypoint for Drive, Docs, Sheets, and Slides work. Use when the user wants to find, fetch, organize, share, export, copy, or delete Drive files, or summarize and edit Google Docs, Google Sheets, and Google Slides through one unified Google Drive plugin. (file: r11/google-drive/SKILL.md)
- google-drive:google-drive-comments: Write, reply to, and resolve Google Drive comments on Docs, Sheets, Slides, and Drive files with evidence-backed location context. Use when the user asks to leave comments, review a file with comments, respond to comment threads, or resolve Drive comments. (file: r11/google-drive-comments/SKILL.md)
- google-drive:google-sheets: Analyze and edit connected Google Sheets with range precision. Use when the user wants to create Google Sheets, find a spreadsheet, inspect tabs or ranges, search rows, plan formulas, create or repair charts, clean or restructure tables, write concise summaries, or make explicit cell-range updates. (file: r11/google-sheets/SKILL.md)
- google-drive:google-slides: Google Slides work for finding, reading, summarizing, creating, importing, template following, visual cleanup, source-deck adaptation, structural repair, and content edits in native Slides decks. (file: r11/google-slides/SKILL.md)
- pdf:pdf: Read, create, inspect, render, and verify PDF files where visual layout matters. Use Poppler rendering plus Python tools such as reportlab, pdfplumber, and pypdf for generation and extraction. (file: r12/pdf/26.630.12135/skills/pdf/SKILL.md)
- presentations:Presentations: Create or edit PowerPoint or Google Slides decks (file: r12/presentations/26.630.12135/skills/presentations/SKILL.md)
- productivity:memory-management: Two-tier memory system that makes Claude a true workplace collaborator. Decodes shorthand, acronyms, nicknames, and internal language so Claude understands requests like a colleague would. CLAUDE.md for working memory, memory/ directory for the full knowledge base. (file: r6/memory-management/SKILL.md)
- productivity:start: Initialize the productivity system and open the dashboard. Use when setting up the plugin for the first time, bootstrapping working memory from your existing task list, or decoding the shorthand (nicknames, acronyms, project codenames) you use in your todos. (file: r6/start/SKILL.md)
- productivity:task-management: Simple task management using a shared TASKS.md file. Reference this when the user asks about their tasks, wants to add/complete tasks, or needs help tracking commitments. (file: r6/task-management/SKILL.md)
- productivity:update: Sync tasks and refresh memory from your current activity. Use when pulling new assignments from your project tracker into TASKS.md, triaging stale or overdue tasks, filling memory gaps for unknown people or projects, or running a comprehensive scan to catch todos buried in chat and email. (file: r6/update/SKILL.md)
- skill-creator: Create new skills, modify and improve existing skills. Use when users want to create a skill from scratch, edit, or optimize an existing skill. (file: r0/skill-

creator/SKILL.md)

- spreadsheets:Spreadsheets: Use this skill when a user requests to create, modify, analyze, visualize, or work with spreadsheet files (`.xlsx`, `.xls`, `.csv`, `.tsv`) or Google Sheets-targeted spreadsheet artifacts with formulas, formatting, charts, tables, and recalculation. (file: r12/spreadsheets/26.630.12135/skills/spreadsheets/SKILL.md)
- template-creator:template-creator: Create or update a reusable personal Codex artifact-template skill. Use when the user invokes \$template-creator or asks in natural language to create a template using, from, or based on an attached Word document, PowerPoint presentation, or Excel workbook, or explicitly asks to edit or update a passed artifact-template skill. Do not use for one-off artifact creation from an existing template. (file: r12/template-creator/26.630.12135/skills/template-creator/SKILL.md)

How to use skills

- Discovery: The list above is the skills available in this session (name + description + short path). Skill bodies live on disk at the listed paths after expanding the matching alias from `### Skill roots`.
- Trigger rules: If the user names a skill (with `\$SkillName` or plain text) OR the task clearly matches a skill's description shown above, you must use that skill for that turn. Multiple mentions mean use them all. Do not carry skills across turns unless re-mentioned.
- Missing/blocked: If a named skill isn't in the list or the path can't be read, say so briefly and continue with the best fallback.
- How to use a skill (progressive disclosure):
 - 1) After deciding to use a skill, the main agent must expand the listed short `path` with the matching alias from `### Skill roots`, then open and read its `SKILL.md` completely before taking task actions. If a read is truncated or paginated, continue until EOF.
 - 2) When `SKILL.md` references relative paths (e.g., `scripts/foo.py`), resolve them relative to the directory containing that expanded `SKILL.md` first, and only consider other paths if needed.
 - 3) If `SKILL.md` points to extra folders such as `references/`, use its routing instructions to identify the files required for the task. The main agent must read each required instruction or reference file itself before acting on it. Do not delegate reading, summarizing, or interpreting skill instructions to a subagent. Subagents may still perform task work when the selected skill allows it.
 - 4) If `scripts/` exist, prefer running or patching them instead of retyping large code blocks.
 - 5) If `assets/` or templates exist, reuse them instead of recreating from scratch.
- Coordination and sequencing:
 - If multiple skills apply, choose the minimal set that covers the request and state the order you'll use them.
 - Announce which skill(s) you're using and why (one short line). If you skip an obvious skill, say why.
- Context hygiene:
 - Progressive disclosure applies to selecting relevant files, not partially reading a selected instruction file. Do not load unrelated references, scripts, or assets.
 - Avoid deep reference-chasing: prefer opening only files directly linked from `SKILL.md` unless you're blocked.
 - When variants exist (frameworks, providers, domains), pick only the relevant reference file(s) and note that choice.
- Safety and fallback: If a skill can't be applied cleanly (missing files, unclear instructions), state the issue, pick the next-best approach, and continue.

</skills_instructions>

<plugins_instructions>

Plugins

A plugin is a local bundle of skills, MCP servers, and apps.

How to use plugins

- Skill naming: If a plugin contributes skills, those skill entries are prefixed with `plugin\_name:` in the Skills list.
- MCP naming: Plugin-provided MCP tools keep standard MCP identifiers such as `mcp\_server\_tool`; use tool provenance to tell which plugin they come from.
- Trigger rules: If the user explicitly names a plugin, prefer capabilities associated with that plugin for that turn.
- Relationship to capabilities: Plugins are not invoked directly. Use their underlying skills, MCP tools, and app tools to help solve the task.
- Relevance: Determine what a plugin can help with from explicit user mention or from the plugin-associated skills, MCP tools, and apps exposed elsewhere in this turn.
- Missing/blocked: If the user requests a plugin that does not have relevant callable

capabilities for the task, say so briefly and continue with the best fallback.
</plugins_instructions>

2. user (2026-07-03T16:13:10.149Z)

AGENTS.md instructions

<INSTRUCTIONS>

Global instructions (all projects)

Repo-freshness convention: git pull BEFORE reading

****Always `git pull` a repo before starting to read it**** ? at session start for the primary working directory, and again for any sibling/local repo the moment work touches it.

- ****Why:**** the owner closes every session on a "clean slate", but multiple parallel slates (sessions/machines) exist ? PRs get merged and master moves between sessions, so the local checkout can silently lag the remote truth. Pull first so ramp-up reads (handoff, docs, code) reflect where the remote actually is.
- ****How (safe form):**** `git pull --ff-only` on the current branch (a plain fetch+ff). Never auto-merge, rebase, or discard local state to make a pull succeed.
- ****If it can't fast-forward**** (diverged branch, dirty tree in the way): do NOT force anything ? report the state to the owner and proceed read-only on what's local, flagging that it may be stale. Uncommitted changes are often a sibling session's or the owner's WIP; never clobber them.

Git branching safety: concurrent sessions / shared checkouts

Multiple sessions ? and spawned/background tasks ? may share ONE working checkout and create branches + commit ****concurrently****, so `git branch` / `git log` can change UNDER you between two commands. (Spawned "fresh worktree" tasks are NOT always isolated.) This has caused a real incident: a sibling commit landed in the gap between a branch-point check and `git checkout -b`, so the new branch rode on top of it and a ****squash-merge** silently absorbed the sibling's work**. Protocol ? do this EVERY time, not only when you suspect a collision:

- ****Branch from the REMOTE, explicitly:**** `git fetch origin` then `git checkout -b <name> origin/<base>` (e.g. `origin/master`). Never assume the current branch is the intended base.
- ****Re-verify right before `git add`/commit:**** `git branch --show-current` + `git log --oneline -1` (HEAD is yours). Before opening a PR, `git log --oneline origin/<base>..HEAD` must list ONLY your commits ? if a sibling commit appears, re-cut the branch from `origin/<base>`.
- ****Stage explicit paths**** (`git add <files>`) ? never `git add -A`/`.`/`-u`, so sibling or untracked files can't ride into your commit.
- ****Serialize repo writes:**** don't start a repo-writing spawned/background task while you hold uncommitted work ? commit or push first. If one may be running, treat the tree + current branch as able to shift, and put this branching-safety reminder into the spawned task's own prompt.

Session-handoff convention

Maintain a living ****session handoff**** for each project and use it to resume context quickly across windows.

- ****Where it lives:****
 - If the project has an auto-memory dir (`~/Codex/projects/<project>/memory/`), keep the handoff as `session-handoff.md` there, and list it under a ****"? Start here**** entry at the top of that project's `MEMORY.md`.
 - Otherwise, keep a `SESSION\_HANDOFF.md` at the repo root.
- ****What it contains**** (keep it to ~1 page ? it POINTS to detailed artifacts, never duplicates them):
 1. ****You are here**** ? current state.
 2. ****Next steps / open threads.****
 3. ****Ramp-up kit**** ? the exact files/docs to read to get current.

4. **Key decisions** ? so they aren't relitigated.
- **At session start:** if a handoff exists, read it before starting substantive work, and use it (plus its ramp-up kit) to get up to speed instead of replaying past conversation.
- **Updating it:** keep it current **automatically** ? refresh at meaningful checkpoints (a PR is pushed/merged, a key decision is made, work shifts phase, or a session is wrapping up), so a restart can ramp up without losing context. Do **not** rewrite it every turn ? only when something worth resuming from changed. The phrases **"update the handoff"** / **"refresh the handoff"** force an immediate full rewrite on demand.
- **Keep it honest:** it reflects real, shipped state ? never aspirational. (See each project's attribution/working-agreement note where present.)

Code review ? pre-LGTM gate (applies to every PR/code review, every project)

Do **not** post **LGTM** or otherwise sign off on a pull request until I have personally **verified** the following ? never trust the PR description's claims, reproduce them locally:

1. **Full test suite is green.** Run the whole suite (e.g. `run_all_tests.py` / `pytest`), not just the changed files.
2. **Install any dependency needed** so the suite actually collects and runs ? do not let modules skip or error for missing deps. (If a check genuinely can't run locally ? GPU/WSL/real-video/etc. ? say so explicitly and note what was and wasn't verified.)
3. **Lint + type checks are clean** ? `ruff check .` and `mypy ...`, matching the repo's CI. Install `ruff`/`mypy` if absent.
4. **Coverage stays up** ? run with coverage and confirm it stays **at or above the repo's floor** (e.g. the CI `--cov-fail-under` value) and does **not regress** versus the base branch.
5. **Review findings are resolved** ? bugs and repo-convention/discipline issues addressed.

LGTM is a **verified** stamp, never a rubber stamp. State plainly what was and wasn't verifiable locally.

</INSTRUCTIONS>

<environment_context>

```
<cwd>C:\Users\avidu\Projects\khelsutra-guru</cwd>
<shell>powershell</shell>
<current_date>2026-07-03</current_date>
<timezone>Asia/Calcutta</timezone>
<filesystem><workspace_roots><root>C:\Users\avidu\Projects\khelsutra-
guru</root></workspace_roots><permission_profile type="disabled"><file_system
type="unrestricted" /></permission_profile></filesystem>
</environment_context>
```

3. user (2026-07-03T16:13:10.178Z)

Can you sync the repositories to remote head and do a thorough analysis of the pending items found in NEXT_STEPS, ROADMAP etc. which were gated on the larger golden corpus and see that we have 15 videos now in the corpus so I want a prioritized list so that we can go towards an alpha launch

4. assistant (2026-07-03T16:13:22.906Z)

I'll first fast-forward the checkout before reading anything, then I'll scan the handoff/planning docs and corpus-related files so the priority list is grounded in the repo's current truth.

5. assistant (2026-07-03T16:13:30.478Z)

The current folder isn't itself a Git repo, so I'm going to locate the actual checkouts underneath it, fast-forward each one safely, and only then read the planning files.

6. assistant (2026-07-03T16:13:37.540Z)

I found this is a workspace containing several repo folders. I'm identifying which ones are actual checkouts now, then I'll fast-forward them one by one and flag any that can't be synced

cleanly.

7. assistant (2026-07-03T16:13:47.861Z)

I'm syncing the five Git checkouts with `git pull --ff-only`. If any branch is dirty or diverged, I'll leave it alone and call that out instead of trying to ?fix? history.

8. assistant (2026-07-03T16:14:14.028Z)

The pulls completed cleanly: two repos advanced, three were already current. Next I'm scanning the current docs and handoffs for the items that were explicitly waiting on corpus size, plus checking where the 15-video corpus is represented.

9. assistant (2026-07-03T16:14:22.222Z)

The repo status is clean after the pulls. I've got the main planning surface now; before synthesizing, I'm also checking the session handoff/memory locations so I don't miss a recent decision that wasn't duplicated into ROADMAP/NEXT_STEPS.

10. assistant (2026-07-03T16:14:31.466Z)

I didn't find an auto-memory handoff under the expected project memory area, so I'm going to use the repo-local planning docs as the source of truth and cross-check them against code/tests/manifests.

11. assistant (2026-07-03T16:14:51.303Z)

The first pass says the pending work clusters around three axes: alpha serving being paused, corpus promotion/evaluation, and the UI operator loop. I'm reading those docs directly now, then I'll separate ?unblocked by 15 videos? from ?still owner/platform-gated.?

12. assistant (2026-07-03T16:15:06.284Z)

The live docs still mark ?grow the golden corpus? as the keystone blocker, but the engine tests already mention a present 15-clip multicourt golden set. I'm now checking manifests, corpus docs, and eval/test code to see what that unlocks concretely rather than just updating a stale bullet.

13. assistant (2026-07-03T16:15:18.731Z)

Good: there are local `output/\*\_golden\_features.json` artifacts and `output/human\_lovo\_manifest.json`, which is exactly the kind of previously missing GPU/corpus material the docs said was gating LOVO/ablation work. I'm counting and summarizing those artifacts next.

14. assistant (2026-07-03T16:15:39.869Z)

There are exactly 15 entries in the local LOVO manifests and 15 matching golden-feature JSONs. One wrinkle: the collector's committed source-video manifest still records only the first six owned MP4s and says uploads are owner-gated, so ?15 present for eval? and ?15 durably promoted into the collector/vault? are not the same state.

15. assistant (2026-07-03T16:15:58.228Z)

I'm going to run the lightweight, corpus-aware eval commands where possible. One thing I'm checking first: the 15-entry manifest has absolute paths, and if any point at an old checkout the ?we have 15? state is less launch-ready than it looks.

16. assistant (2026-07-03T16:16:10.244Z)

The manifest is stale for this workspace: all 15 absolute trajectory/label paths currently fail as written. The feature JSONs are still usable for feature-level eval, but any manifest-driven training/serve-contrast command needs a path-rebased manifest before it can be trusted.

17. assistant (2026-07-03T16:16:22.555Z)

The local copies needed to rebase the manifest are present in this workspace, so the stale absolute paths are fixable. For the analysis, I'm first using the feature-level commands that don't need that rebase.

18. assistant (2026-07-03T16:16:40.719Z)

The first eval result is telling: `fusion\_compare` only loaded 6 of the 15 feature files, so the person-fusion machinery is still effectively n=6. The newer 9 are likely ?windowing-only? feature schema and need regeneration/upconversion before they can answer the P3 fusion question. Also, the ablation CLI currently fails at import with a circular import, which makes R2 promotion less turnkey than the docs imply.

19. assistant (2026-07-03T16:16:49.872Z)

So the actual state is split: 15 windows/GT artifacts exist, but only 6 have full shuttle+person+audio candidate features. That changes the alpha roadmap: first priority is making the 15-video corpus reproducible and feature-complete, not immediately trusting the higher-level ?15 videos? headline.

20. assistant (2026-07-03T16:17:01.535Z)

I'm rebasing the manifest into a temp file outside the repo so I can exercise manifest-driven eval without editing tracked or ignored project files. That should tell us whether the 15 trajectories/labels are actually usable end to end from this workspace.

21. assistant (2026-07-03T16:17:26.568Z)

The 15-video manifest works once rebased. Gate-A's proposal-side lift is real-ish but served-side regresses, so it should not become the served default for alpha. I'm running the Gen-0 harness now against the rebased n=15 manifest to see whether the larger corpus changes the model/ship-gate picture.

22. assistant (2026-07-03T16:19:11.998Z)

Gen-0 on all 15 now runs end to end from a rebased manifest. It clears the old n=6 heuristic floor, but mean LOVO F1 is 0.551 and the comparison floor/sign test is still anchored to the old n=6 baseline, so the alpha decision needs an n=15 baseline refresh before anyone can claim lift.

23. assistant (2026-07-03T16:19:21.144Z)

Next I'm looking for an existing n=15 heuristic/tolF1 baseline path. If it's already in the repo, that becomes a concrete next action; if not, the priority is to make that baseline artifact first.

24. assistant (2026-07-03T16:19:28.848Z)

The old floor artifact is explicitly n=6, so I'm recomputing the heuristic LOVO floor on the rebased 15-video manifest. This is the number the Gen-0 result should be judged against, not the stale baseline.

25. assistant (2026-07-03T16:22:45.272Z)

The n=15 heuristic recalibration did not finish inside a 3-minute window. I'm not going to leave it half-claimed; I'll mark it as an explicit alpha-priority action unless I can find a lighter existing resweep artifact for the same question.

26. assistant (2026-07-03T16:22:49.330Z)

There is already an n=13 resweep showing threshold tuning had moved from ?retrain? to ?measure/regression? territory. With 15, the same resweep should be rerun, but the bigger immediate gap is not cap tuning; it's feature completeness and baseline freshness.

27. assistant (2026-07-03T16:22:55.001Z)

I'm checking the nightly regression runner now because it may already have the current default-vs-milestone R2 summary on cached trajectories. If it needs the stale manifest paths, I'll avoid mutating repo state and treat this as a required re-snapshot step.

28. assistant (2026-07-03T16:22:59.553Z)

The runner accepts a manifest override, so I can run it against the rebased n=15 corpus without touching the committed baseline.

29. assistant (2026-07-03T16:23:15.689Z)

The offline nightly runner gives us a useful n=15 R2 snapshot even though the frozen baseline is stale: current served/default toIF1 is 0.364, baseline-off is 0.138, and the run correctly says the baseline must be refreshed before it can be a gate. That is exactly the sort of alpha-blocking hygiene we should do before launch claims.

30. assistant (2026-07-03T16:23:39.734Z)

I'm collecting exact heads/line references now so the final list is auditable rather than just vibes in a trench coat. After that I'll give you the ranked alpha plan.

31. assistant (2026-07-03T16:23:43.716Z)

One of my reference-gathering commands had a PowerShell pipe typo; rerunning just the repo-head table cleanly.

32. assistant (2026-07-03T16:23:53.636Z)

I'm doing one final pass over the root audit summary because it often captures cross-repo leftovers that the per-repo roadmaps don't surface prominently.

33. assistant (2026-07-03T16:23:59.788Z)

The root audit agrees with the same pattern: code-health remediation is done; remaining items are corpus upload scope, GCS eval/litmus plumbing, hosted deployment, and product backlog. That reinforces the alpha priority order.

34. assistant (2026-07-03T16:24:01.161Z)

I'm going to stop tool work here and synthesize. No repo files were edited; the only generated files were temp analysis artifacts under `%TEMP%`.

35. assistant (2026-07-03T16:25:20.864Z)

Synced the Git repos first. Current heads are clean and at remote:

```
| repo | branch | HEAD |
|---|---:|---:|
| `badminton-highlight-indexer` | `master` | `e4914d1` |
| `khelsutra` | `master` | `b0f70a5` |
| `rally-annotator` | `main` | `ef9e088` |
| `sports-data-collector` | `master` | `bfc014f` |
| `sports-obsreport` | `main` | `a0c7252` |
```

`khelsutra-screencast` is not a Git checkout.

****Bottom Line****

The 15-video corpus changes the state from ?we need more labels? to ?we need to make the corpus reproducible, feature-complete, and gate-ready.? I would not treat the 15-video headline as alpha-ready by itself.

The most important split: an invite-only alpha can move forward as a hosted, authenticated ?upload video ? get reel? product, but the owned rally-detection/model launch is still explicitly no-go until the ship gate and WASB provenance are resolved. See [engine NEXT_STEPS](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/NEXT_STEPS.md:8).

****What I Verified****

- Golden registry says `output/human\_lovo\_manifest.json` is now `n = 15`: [GOLDEN_VIDEOS.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/data_pipeline/GOLDEN_VIDEOS.md:28).
- Local manifest has 15 videos / 595 rallies, but all absolute paths were stale for this workspace. I rebased a temp manifest and confirmed the trajectories/labels exist.
- Only 6 of the 15 `\*\_golden\_features.json` files are full fusion schema. The newer 9 are windowing-only, so person/audio fusion analysis is still effectively n=6.
- `fusion\_compare` on usable full-fusion files: n=6, person fusion ?F1 `-0.021`, not promotable.
- `serve\_contrast` on rebased n=15: proposal Gate A mean ?F1 `+0.0287`, but served mean ?F1 `-0.0328`; do not promote to served default.
- Gen-0 harness on rebased n=15: mean LOVO F1 `0.551`, `ship=False`; blockers remain ship-gate calibration and WASB C3 provenance. Harness hard-codes those blockers at [harness.py](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/training/gen0/harness.py:203).
- Nightly R2 runner on n=15 reports current/default tolF1 `0.3636`, baseline-off tolF1 `0.1382`, but exits stale because the frozen baseline must be refreshed.
- `ablation` CLI currently fails from a circular import between [ablation.py](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/ablation.py:530) and [ablation_pdf.py](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/ablation_pdf.py:17).

****Prioritized Alpha List****

1. **Stand up shareable alpha serving.**

This is still the biggest launch blocker: `khelsutra` says alpha serving is paused pending Cloud Run: [NEXT_STEPS.md](C:/Users/avidu/Projects/khelsutra-guru/khelsutra/NEXT_STEPS.md:13). Do Cloud Run/GPU host, edge auth ON, health endpoint, upload/process/job polling, reel download.

2. **Make the 15-video corpus reproducible.**

Fix or regenerate a portable manifest, update the golden registry to enumerate all 15 cleanly, and promote MD5/video records into the collector/vault path. Collector still records only six owned source MP4s with uploads owner-gated: [golden_source_videos.json](C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector/data/golden_source_videos.json:4).

3. **Refresh the n=15 quality gate.**

Re-snapshot nightly regression intentionally, fix the ablation CLI, recompute the heuristic n=15 floor, then ratify the alpha metric. The docs? key pending items are ship-gate target and R2 tolF1 gate: [NEXT_STEPS.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/NEXT_STEPS.md:23).

4. **Feature-complete the newer 9 videos.**

Regenerate full shuttle/person/audio features for all 15 before making any fusion/TemporalMaxer decisions. Right now P3 fusion is not actually using the larger corpus.

5. **Run full-corpus training as shadow-only.**

The n=15 Gen-0 run is useful, but not shippable. Next useful model work is M0.4/TemporalMaxer after the gate refresh, not before: [NEXT_STEPS.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/NEXT_STEPS.md:29).

6. **Un-defer corpus flywheel P10/P11 only after the above.**

The collector bridge is done, but corpus promotion and train/re-eval jobs remain deferred: [STUDIO_MEDIA_LIFECYCLE.md](C:/Users/avidu/Projects/khelsutra-guru/khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md:163). This becomes alpha-plus: valuable for learning from testers, not required for the first controlled alpha if manual promotion works.

7. **Deprioritize for alpha: multisport, Google Drive API storage, audio cue work, repo rename, and broad model claims. Also do not promote Gate A served-default or person fusion based on the current measurements.**

No repo files were edited; temp analysis artifacts were written only under `%TEMP%`.

36. user (2026-07-03T16:28:50.601Z)

Can you make this into a project doc which we can iterate and resolve? Just the docs PR with the concrete next steps would be good

37. assistant (2026-07-03T16:29:07.577Z)

I'll turn the analysis into a concrete project doc and publish it as a docs-only PR. I'm going to use the documentation workflow for the doc shape and the GitHub PR workflow for the branch/commit/push bits.

38. assistant (2026-07-03T16:29:26.992Z)

I'm going to place this in the engine repo as a cross-repo alpha readiness tracker, since that's where the corpus, ship-gate, ROADMAP, and NEXT_STEPS source-of-truth already live. I'll link out to `kheIsutra` and collector items rather than duplicating their full docs.

39. assistant (2026-07-03T16:29:31.938Z)

`gh` is available and authenticated, and the engine repo is up to date. I'm creating a fresh branch directly from `origin/master` now, then I'll make only docs changes.

40. assistant (2026-07-03T16:29:43.783Z)

Branch is cut from the remote head. I'm reading the local docs index/template so the new tracker fits the repo's current documentation style instead of becoming a lonely orphan file.

41. assistant (2026-07-03T16:30:02.046Z)

I'm going to add a new tracked doc, `docs/ALPHA\_LAUNCH\_READINESS.md`, then index it in `docs/README.md` and `docs/DOC\_STATUS.md`. The doc will keep the launch/product work separate from the owned-model quality gate so future iterations don't blur those two decisions.

42. assistant (2026-07-03T16:31:32.685Z)

The tracker doc is in place and indexed. I'm reviewing the diff now for accuracy and doc style before committing.

43. assistant (2026-07-03T16:31:49.040Z)

The new file has one inherited non-ASCII separator in the header; I'm cleaning that up so the freshly added doc stays ASCII while the existing docs keep their current style.

44. assistant (2026-07-03T16:32:14.725Z)

I'm adding a small pointer from the live `NEXT\_STEPS.md` P0 item to the new tracker, so future sessions starting from the old roadmap land on the concrete checklist.

45. assistant (2026-07-03T16:32:37.299Z)

The doc is now also linked from live `NEXT\_STEPS.md`. I'm running the lightweight validation pass: whitespace check plus final status/diff review.

46. assistant (2026-07-03T16:32:45.170Z)

Whitespace check is clean; the only warning is Git's existing Windows line-ending normalization notice. I'm re-verifying the branch/HEAD now, then I'll stage only the four docs paths and commit.

47. assistant (2026-07-03T16:32:54.607Z)

The docs commit is created. I'm doing the branch-safety check before pushing: current branch, HEAD, and `origin/master..HEAD` should show only this docs commit.

48. assistant (2026-07-03T16:33:02.902Z)

The branch contains only the docs commit. I'm pushing it now, then I'll open a draft PR.

49. assistant (2026-07-03T16:33:10.255Z)

Branch is on GitHub. I'm checking for the GitHub PR tool first; if the connector isn't exposed cleanly here, I'll use `gh pr create` as the fallback.

50. assistant (2026-07-03T16:33:33.540Z)

The GitHub connector returned a 404 for PR creation, likely a connector/repo visibility mismatch, so I'm using the authenticated `gh` fallback as planned.

51. assistant (2026-07-03T16:33:53.102Z)

PR is open. I'm doing one tiny follow-up so the tracker's A0 row points at the actual PR number instead of the placeholder `this PR`, then I'll amend and update the branch.

52. assistant (2026-07-03T16:34:06.103Z)

The small PR-number edit is clean. I'm amending the docs commit and force-with-lease pushing the updated branch.

53. assistant (2026-07-03T16:34:16.080Z)

Branch is updated. I'm doing the final clean-state check and PR metadata readback.

54. assistant (2026-07-03T16:34:25.664Z)

Done. I created a docs-only draft PR:

[#451 - [codex] add alpha launch readiness tracker](https://github.com/Khelsutra/badminton-highlight-indexer/pull/451)

It adds [ALPHA_LAUNCH_READINESS.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/ALPHA_LAUNCH_READINESS.md), indexed from [docs/README.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/README.md) and [docs/DOC_STATUS.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/DOC_STATUS.md), with a pointer from [NEXT_STEPS.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/NEXT_STEPS.md).

The doc captures the concrete alpha sequence: hosted serving, corpus portability, n=15 baseline refresh, ablation/fusion reruns, ship-gate/C3 decisions, and final GO/NO-GO. Validation: `git diff --check` and `git diff --check HEAD^..HEAD`. No runtime tests run because this is docs-only.

55. user (2026-07-03T16:38:26.449Z)

you may convert this from a draft to a PR for review. I will ping you when I have comments

56. assistant (2026-07-03T16:38:34.579Z)

I'll mark #451 ready for review now and verify GitHub reflects the change.

57. assistant (2026-07-03T16:38:42.637Z)

Done. PR #451 is now ready for review:

[#451 - [codex] add alpha launch readiness tracker](https://github.com/Khelsutra/badminton-highlight-indexer/pull/451)

58. user (2026-07-03T16:40:38.442Z)

BTW F:\ drive is connected, can you cross check if it has the right information as mentioned in the repo? Or it wasnt mentioned?

59. assistant (2026-07-03T16:40:52.745Z)

I'll check that against the repo records directly. Since this touches the repo state, I'll fast-forward the relevant checkouts first, then compare the `F:\` files against the committed source-video manifest.

60. assistant (2026-07-03T16:41:02.365Z)

Both relevant repos are already up to date. I'm scanning for `F:\` references and then I'll verify the actual files on the drive.

61. assistant (2026-07-03T16:41:09.967Z)

It is mentioned: the collector repo has a committed `data/golden\_source\_videos.json` with six `F:\` source paths, sizes, timestamps, and MD5s; the engine docs also mention the broader `F:\` archive as the source-video gap. I'm checking whether the mounted drive matches those records now.

62. assistant (2026-07-03T16:41:21.788Z)

The six `F:\` files named in the committed manifest all exist and their byte sizes match exactly. I'm doing the slower MD5 pass now so we can verify content identity, not just filenames and sizes.

63. assistant (2026-07-03T16:43:24.527Z)

The MD5s all match for the six files in the collector manifest. I'm doing one more pass for the broader engine doc mention, which names the `F:\` archive root and specifically calls out two additional clips (`GX010137`, `GX010141`) alongside the original six.

64. assistant (2026-07-03T16:43:53.528Z)

Yes, it **was mentioned** in the repo, and the mounted `F:\` drive matches the committed records.

I checked [golden_source_videos.json](C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector/data/golden_source_videos.json:5), which records six owned golden source MP4s under:

`F:\[Khelsutra] GoPro Hero Black 12\GoPro Recorded Videos\Training\Golden Labelled`

Result: all six files exist, byte sizes match, and MD5 hashes match exactly:

- `adarsh\_avi\_singles`
- `kushagra\_singles`
- `largetest\_doubles`
- `gbaaddy`
- `testlarge\_short`
- `mahadevpura\_singles`

The repo also mentions the broader `F:\` archive in [DATA_IN_GCS.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/DATA_IN_GCS.md:139), specifically saying F has the original six plus `GX010137` and `GX010141` as full MP4s. I found those too under:

`F:\[Khelsutra] GoPro Hero Black 12\GoPro Recorded Videos\Badminton - Boxhill - 17 June`

So the short version: **F:** is real, mounted, and consistent with the repo records for the files that are explicitly documented there.** It does not mean all 15 eval videos are recorded in the collector manifest yet; that manifest currently documents the original six source MP4s, while the engine docs say the remaining video/proxy migration is still part of the larger corpus gap.

65. user (2026-07-03T16:44:43.861Z)

so just 6 and not the other golden labeled videos?

66. assistant (2026-07-03T16:44:56.801Z)

I'll do a quick exact-name search across the **F:** archive for all 15 manifest video names, so we can separate ?documented in the collector manifest? from ?present somewhere on F.?

67. assistant (2026-07-03T16:45:10.687Z)

That command hit a PowerShell pipe parsing quirk; rerunning the same search with the results collected first.

68. assistant (2026-07-03T16:45:34.465Z)

Not just six on the drive.

The distinction is:

- **Committed collector integrity manifest:** only **6** videos are recorded in **sports-data-collector/data/golden_source_videos.json**, with byte size + MD5. I verified all six match exactly.
- **Actually present on F:\:** I found MP4s for **all 15 golden eval videos** by name, including the newer golden-labeled clips:
mahadevpura-1, **mahadevpura-2**, **GX010128**, **GX020094**, **GX030094**, **Badminton BXH 2**, **GX010137**, **GX010141**, and **Boxhill Doubles**.

So the repo is under-documenting the current state: the **six original source MP4s** are formally tracked and hash-verified, while the **other nine** appear to be present on **F:** but do not yet have the same committed MD5/source manifest treatment.

That maps cleanly to the alpha doc's A3 item: reconcile the collector/vault source-video records with the full 15-video eval corpus.

69. user (2026-07-03T16:49:19.372Z)

Can you ptal at the review comments on#451 - [codex] add alpha launch readiness tracker and respond accordingly

70. assistant (2026-07-03T16:49:30.157Z)

I'll inspect the PR review threads, make any doc fixes they call for, and reply/resolve where appropriate. I'm using the GitHub review-comment workflow here so I can distinguish unresolved review threads from ordinary PR chatter.

71. assistant (2026-07-03T16:49:44.203Z)

First I'm syncing the local PR branch and confirming GitHub auth; then I'll pull thread-aware review data instead of relying on the flattened comment view.

72. assistant (2026-07-03T16:49:49.709Z)

The branch is up to date and **gh** is authenticated. I found the bundled review-thread fetcher, so I'm using that next to get unresolved inline comments with file/line anchors.

73. assistant (2026-07-03T16:50:08.195Z)

There are six unresolved inline threads, all actionable and doc-only. I'm folding them into the tracker now: table rendering, D13 qualification, M11 cross-link, A6 dependency, reproducible command, path normalization, analysis tags, A0 status, and the observability note from the review summary.

74. assistant (2026-07-03T16:50:37.551Z)

The review fixes are applied locally. I'm checking for lingering path inconsistency/table issues and whitespace before amending the PR branch.

75. assistant (2026-07-03T16:50:49.062Z)

The substantive diff matches the review notes. I'm doing a simpler grep and a quick table-pipe count now because the first regex used a PCRE-only lookbehind.

76. assistant (2026-07-03T16:50:54.851Z)

The table rows all have the expected pipe count now, and the remaining grep hits are the corrected forms. I'm amending the existing docs commit and pushing the updated PR branch.

77. assistant (2026-07-03T16:51:06.173Z)

The PR branch is updated. I'm rereading the review-thread state now; depending on GitHub's outdated-thread behavior, I'll either leave them naturally outdated or reply with a concise ?addressed in latest push.?

78. assistant (2026-07-03T16:51:19.522Z)

All six inline threads are now outdated on GitHub after the push, which is what we want for line-level doc fixes. I'm adding one PR comment summarizing the exact responses instead of marking the reviewer's threads resolved on their behalf.

79. assistant (2026-07-03T16:51:34.211Z)

The response comment is posted. I'll do one final local/PR state check so I can hand you back a clean summary.

80. assistant (2026-07-03T16:51:41.613Z)

Addressed the review comments on [PR #451](https://github.com/Khelsutra/badminton-highlight-indexer/pull/451) and pushed the updated branch.

Changes are in commit `41ce99c`:

- Fixed the A12 table pipe issue.
- Qualified the `D13` reference to the `khelsutra` Studio lifecycle doc.
- Cross-linked A12 to `COMPUTE\_DECOUPLED\_SERVING` M11.
- Removed the unnecessary A4 dependency from A6.
- Tagged D3/D4 as `[verified]` / `[analysis]`.
- Made the Gen-0 command reproducible.
- Normalized the `GOLDEN\_VIDEOS.md` path.
- Set A0 to `DONE`.
- Added observability/job-failure triage to A13.

I also posted a PR comment summarizing the response. Validation: `git diff --check` plus a pipe-count check over the Section 7 tracker table.

81. user (2026-07-03T17:08:23.260Z)

The PR is merged and A3 is picked up by another agent. What do you think you can pick up right now?

82. assistant (2026-07-03T17:08:32.595Z)

I'll sync the engine repo back to remote head first so I'm not reasoning from the now-merged PR branch. Then I'll map the A-list against what's unblocked and non-conflicting with the other agent on A3.

83. assistant (2026-07-03T17:08:44.479Z)

Master is already current. Since A3 is occupied, I'm checking the tracker rows and the nearby code/doc anchors to choose something useful that won't collide.

84. assistant (2026-07-03T17:09:00.975Z)

I'd pick up **A2** next.

It's the cleanest, non-conflicting piece right now: **make the n=15 golden manifest portable/reproducible and add a smoke check that fails loudly when labels/trajectories are missing**. It stays inside `badminton-highlight-indexer`, does not overlap with the other agent's A3 collector/vault work, and it unblocks A4/A5/A6/A7.

Concrete scope I'd take:

- Add a small manifest validation/rebasing utility or testable helper.
- Make stale absolute paths visible as a hard failure, not silent skips.
- Add/update tests around `all manifest labels + trajectories resolve`.
- Update `ALPHA_LAUNCH_READINESS.md` A2 status and notes in the same PR.

A smaller quick win is **A6 first half**: fix the `backend.eval.ablation` circular import. But I'd prefer A2 first because A6's actual n=15 run should sit on a trustworthy manifest surface. A1 is bigger and owner/platform-gated; A4/A5 are cleaner after A2; A7 likely needs GPU/data regeneration; A3 is already occupied.

85. user (2026-07-03T17:10:38.999Z)

Please go ahead and start working. Both the GPU and Gemini API calls are free for you to use even running a GCP L4 machine with a budget cap of 300\$ is approved. The gemini key will be at `G:\My Drive\gemini.api_key.txt` and the gcloud access to `avi.dullu@gmail` should be open while the ability to schedule GPU machines should also be active

86. assistant (2026-07-03T17:10:52.351Z)

I'll start with A2, staying out of the collector/vault lane that another agent has. I'll sync and branch from `origin/master`, then make the manifest portability/smoke-check change in the engine repo.

87. assistant (2026-07-03T17:10:58.805Z)

The repos are current. I'm cutting a fresh branch from `origin/master` now and then I'll inspect the eval manifest loaders so the fix lands in the right seam.

88. assistant (2026-07-03T17:11:09.860Z)

Branch is ready. I'm reading the manifest-loading paths and nearby tests to decide whether this should be a shared helper, a CLI, or just a test guard.

89. assistant (2026-07-03T17:12:30.445Z)

I'll pick up A2 first: making the n=15 golden manifest portable and adding a loud smoke check for missing labels/trajectories. I'm going to re-sync the repos before reading further, then keep the edit scoped to `badminton-highlight-indexer` so we don't collide with the A3 agent.

90. assistant (2026-07-03T17:12:40.617Z)

Both repos are fresh. Next I'm checking the current branch and the manifest/eval entry points so the patch lands on the right base and doesn't accidentally rope in someone else's work.

91. assistant (2026-07-03T17:12:49.627Z)

The branch is clean and based on `origin/master`. I've got enough signal to avoid the broader A3/source-video area; now I'm reading the loaders and the golden-regression test paths that currently skip or silently drop stale manifest entries.

92. assistant (2026-07-03T17:12:59.923Z)

The current pain is exactly where expected: tests skip when stale absolute paths don't exist, while `rally\_seg\_eval` silently leaves videos unscored if trajectories are missing. I'm going to add a small resolver/checker module and thread it into the owner-box smoke paths without changing the collector/source-video manifest work.

93. assistant (2026-07-03T17:13:16.995Z)

I'm keeping the implementation boring on purpose: one resolver module, unit tests with temporary manifests, and small call-site changes where stale paths currently matter. That should make A2 reviewable without dragging in GPU/GCP or the source-video provenance work.

94. assistant (2026-07-03T17:13:34.210Z)

I'm also checking the real manifest shape before editing. The resolver will support today's flat list format and deliberately fail if a required `trajectory` or `labels` path can't be resolved, instead of letting the corpus shrink quietly.

95. assistant (2026-07-03T17:13:44.036Z)

The local manifest has all 15 entries but still points at the older `Annotation Setup/Collect/...` layout. The sibling `Annotation Setup/Trajectories`, `Annotation Setup/Golden Labelled`, and repo `output/` directories have the files, so a deterministic basename-based rebase is enough for A2.

96. assistant (2026-07-03T17:14:43.705Z)

I'm going to make the resolver reusable for the main flat golden-manifest consumers: rally segmentation/nightly, served Gate-A contrast, and Gen-0 training. That way users can either run the smoke checker directly or just pass the old manifest to the existing commands on this workspace and get deterministic rebasing.

97. assistant (2026-07-03T17:15:28.942Z)

I'll update the tracker too, but only to mark A2 as the active implementation PR, not as merged. The code will carry the actual smoke-check command so the docs are useful the next time someone sits down with the corpus.

98. assistant (2026-07-03T17:15:34.561Z)

Time to edit. I'm adding `backend.eval.golden\_manifest` as the portable-manifest layer, then I'll wire the existing commands to use it and add focused tests around stale absolute paths, missing files, and writing a local resolved manifest.

99. assistant (2026-07-03T17:16:36.273Z)

The resolver module is in place. I'm now swapping the brittle direct `json.load` calls in the key quality paths for the strict loader, using trajectory-only resolution where labels aren't consumed and labels+trajectories where they are.

100. assistant (2026-07-03T17:16:43.710Z)

I'm broadening this just enough so the commands named in the launch tracker don't require a hand-written rebased temp file anymore. This is still local path resolution only, not corpus promotion or MD5/source-video bookkeeping.

101. assistant (2026-07-03T17:16:57.742Z)

There are a few sibling eval CLIs that also consume the same flat `{name, trajectory, labels}` manifest. I'm wiring those too so ?portable manifest? means one behavior across the quality tools rather than a surprise only in the nightly path.

102. assistant (2026-07-03T17:17:12.828Z)

The owner-box served litmus no longer needs to hand-filter stale paths. I'm changing its setup to use the same resolver, while still skipping cleanly on machines that don't have the private corpus.

103. assistant (2026-07-03T17:17:31.354Z)

I'm adding tests that mimic the real failure mode: an old `C:/.../Annotation Setup/Collect/...` manifest, files in the moved workspace layout, plus an explicit missing-file case. That gives reviewers confidence without needing the private corpus in CI.

104. assistant (2026-07-03T17:18:06.169Z)

I'm documenting the new smoke command right where the launch tracker points people next, and adding a short note to the golden-video registry so stale `Collect/...` paths don't send future agents on a goose chase.

105. assistant (2026-07-03T17:18:30.589Z)

The patch is assembled. I'm doing a quick static pass now: check the diff for accidental scope creep, run the focused tests, and then try the real n=15 smoke command against the local corpus.

106. assistant (2026-07-03T17:18:39.741Z)

One small wrinkle: the new files are untracked, so the first diff stat didn't include their size yet. I'm checking lint/format next; if ruff complains, I'll tighten the long imports and doc lines before running the tests.

107. assistant (2026-07-03T17:18:49.968Z)

Focused tests are green. Ruff found only one import-format nit in the served litmus; I'm patching that and then I'll run the real manifest check plus a smoke pass through the existing commands.

108. assistant (2026-07-03T17:19:18.829Z)

The real n=15 path check passes and rebases all 30 required paths from the stale manifest. I'm now proving the old manifest works directly with the two analysis commands that previously needed a temporary rebased copy.

109. assistant (2026-07-03T17:19:50.152Z)

The existing commands now accept the stale manifest directly. The nightly still returns stale-baseline status, as expected for A4, but it scored all 15 instead of silently shrinking. I'm running the owner-box served litmus too, since the resolver makes those real-corpus tests active here.

110. assistant (2026-07-03T17:20:26.285Z)

Served litmus, Gen-0 tests, and the real rally eval all pass with the stale manifest path. I'm checking the tree for generated artifacts and then I'll run the repo's broader verification lane that's reasonable for this scoped PR.

111. user (2026-07-03T17:23:09.210Z)

can you please pause after A2 PR is up for review and let me do some reassessment

112. assistant (2026-07-03T17:23:20.954Z)

Got it. I'll take A2 to a reviewable PR, then stop and leave the next move to you. The broad suite exposed a few synthetic fusion-golden tests that need dummy manifest files now that the loader is strict, so I'm tightening those tests rather than weakening the A2 check.

113. assistant (2026-07-03T17:23:27.577Z)

The failures are all the same pattern: test manifests had placeholder paths because ``extract_video`` was monkeypatched. Since A2 deliberately validates manifests before long runs, I'm making those fixtures create tiny placeholder trajectory/label files.

114. assistant (2026-07-03T17:23:41.810Z)

The fixture patch is in. I'm rerunning the exact failing slices first, then the full suite again so the PR isn't carrying a surprise.

115. assistant (2026-07-03T17:23:56.821Z)

The failing slices are fixed. CI's actual gates include coverage and mypy, so I'm running those now with ``ruff check .``; this will take a couple of minutes, but it's the right pre-PR bar for this repo.

116. assistant (2026-07-03T17:27:57.574Z)

All CI-style gates are green now: coverage suite, ruff, and mypy. I'm doing the final diff/whitespace check, then I'll commit only the A2 files and push a PR for review.

117. assistant (2026-07-03T17:28:13.806Z)

The diff is clean. I'm following the repo's concurrency rules now: re-check branch/HEAD, stage explicit paths only, then commit and push this A2 branch.

118. assistant (2026-07-03T17:28:17.063Z)

HEAD is the merged tracker commit and there are no local commits yet, so the branch is correctly isolated. I'm staging only the files touched for A2.

119. assistant (2026-07-03T17:28:26.829Z)

Staged paths are explicit. I'm doing one last staged diff summary and branch check before committing.

120. assistant (2026-07-03T17:28:30.169Z)

Everything staged is A2-scoped. Committing now.

121. assistant (2026-07-03T17:28:39.268Z)

Commit is made. I'm fetching remote and verifying the branch contains only this A2 commit above ``origin/master`` before I push/open the PR.

122. assistant (2026-07-03T17:28:46.003Z)

Remote is fetched; the branch is one commit ahead. I'm checking the exact commit list, then I'll push.

123. assistant (2026-07-03T17:28:49.034Z)

The branch contains only the A2 commit over master. Pushing it now and then I'll open the PR.

124. assistant (2026-07-03T17:28:58.422Z)

The branch is pushed. Since this is a publish/PR step, I'm checking the repo's GitHub PR workflow instructions before creating the PR body.

125. assistant (2026-07-03T17:29:09.563Z)

The publish workflow says to verify `gh` and auth, then prefer the GitHub connector if available. I'll check both and create the PR with the validation details.

126. assistant (2026-07-03T17:29:13.208Z)

GitHub auth is good. I'm opening the PR now with a concise body: what changed, why, and the exact checks/results.

127. assistant (2026-07-03T17:29:34.442Z)

The connector couldn't create the PR for this private repo (404 from the API path), so I'm falling back to `gh pr create`, which is exactly what the publish workflow recommends in that case.

128. assistant (2026-07-03T17:30:02.729Z)

PR #452 is open for review. I'll do one final readback/status check and then pause as requested.

129. assistant (2026-07-03T17:30:12.271Z)

A2 PR is up for review and not draft: [#452 - [codex] add portable golden manifest check](https://github.com/Khelsutra/badminton-highlight-indexer/pull/452)

What landed:

- Portable golden manifest resolver/checker: `backend.eval.golden\_manifest`
- Existing quality commands can now use the stale `output/human\_lovo\_manifest.json` directly
- Loud path smoke check for missing trajectories/labels
- Docs updated with the A2 command
- A2 marked `IN PROGRESS` in the tracker pending PR review/merge

Validation:

- `ruff check .`
- `mypy backend training`
- `python run\_all\_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70`
 - `1605 passed, 6 skipped`, coverage `85.44%`
- Real n=15 smoke check passed: 15 videos, 30 required paths present
- `serve\_contrast` and nightly regression both run from the stale manifest directly now

No GPU/Gemini/GCP spend used for this one. Pausing here for your reassessment.

130. user (2026-07-03T17:41:04.999Z)

Can you please address the review comments?

131. assistant (2026-07-03T17:41:13.127Z)

I'll take the review loop on #452 now. I'm first fast-forwarding the local PR branch per your repo-freshness rule, then I'll inspect the unresolved review threads and patch only what the reviewer asked for.

132. assistant (2026-07-03T17:41:18.518Z)

Using the GitHub review-comments workflow for this turn so I can distinguish unresolved actionable threads from general PR noise.

133. assistant (2026-07-03T17:41:29.300Z)

The skill says to use thread-aware GitHub reads rather than flat comments, because

resolved/outdated state matters. I'm resolving #452 from the current branch and pulling the review threads next.

134. assistant (2026-07-03T17:41:33.963Z)

The flat PR view shows no top-level comments, so this is likely inline/threaded review feedback. I found the bundled thread fetcher and I'm reading its interface before running it.

135. assistant (2026-07-03T17:41:39.873Z)

The fetcher resolves the current branch's PR and includes `isResolved`/`isOutdated`, which is what we need. I'm running it now and then I'll cluster only unresolved actionable comments.

136. assistant (2026-07-03T17:42:01.883Z)

There are four actionable items: wire `fusion\_audio`, preserve manifest error details in the served-litmus skip reason, harden ambiguous basename matches, and dedupe `DEFAULT\_MANIFEST`. I'm going to implement all four; I'll leave the "atomic JSON helper" note alone as a broader cleanup because it's explicitly non-blocking.

137. assistant (2026-07-03T17:42:40.181Z)

I'm tightening the resolver rather than just satisfying the superficial comment: trajectories won't fall back to `output/` by default, ambiguous basename matches will become explicit manifest issues, and the CLI will use the resolver's own rebase records instead of recomputing them.

138. assistant (2026-07-03T17:43:13.586Z)

The core hardening is patched. I'm checking the updated resolver because this is the place most likely to have type/signature fallout after returning rebase records.

139. assistant (2026-07-03T17:43:34.437Z)

I'm also adopting the reviewer's constant-ownership nit: `golden\_manifest` owns the canonical `output/human\_lovo\_manifest.json`, and `rally\_seg\_eval` will import that instead of duplicating it.

140. assistant (2026-07-03T17:43:49.005Z)

Next I'm fixing the served-litmus skip signal so a missing private corpus still skips in CI, but an owner-box path break reports the resolver's actual diagnostics.

141. assistant (2026-07-03T17:43:59.061Z)

I'm updating and extending the resolver tests now: they'll assert the new rebase records, ambiguous labels are rejected, and trajectory fields don't accidentally resolve from `output/`.

142. assistant (2026-07-03T17:44:30.945Z)

The code changes are in place. I'm running the focused tests first so any review-comment fallout is easy to isolate before I do the broader gate.

143. assistant (2026-07-03T17:45:10.318Z)

The new trajectory fallback test caught that `manifest\_dir` was still effectively `output/` for stale absolute trajectory basenames. I'm narrowing that fallback to labels only; relative trajectory paths still resolve via the explicit manifest-relative candidate.

144. assistant (2026-07-03T17:45:20.651Z)

That was a useful little trapdoor. I'm rerunning the focused set and the real n=15 smoke check now.

145. assistant (2026-07-03T17:46:00.845Z)

Focused checks are green and the real corpus still resolves cleanly. I'm running the full PR gates again now, plus a quick real-command smoke after the resolver hardening.

146. assistant (2026-07-03T17:49:43.936Z)

The full gates are green. I want one direct regression test for the `fusion\_audio` comment too, so I'm adding a small CPU-only augment test that proves stale label paths are rebased before deriving the MP4 path.

147. assistant (2026-07-03T17:49:53.175Z)

This test will exercise the exact reviewed behavior without invoking ffmpeg: a stale manifest, rebased labels, derived MP4 path, audio feature write.

148. assistant (2026-07-03T17:50:13.151Z)

The direct `fusion\_audio` regression test is in. I'm rerunning the focused set and lint, then I'll only rerun the full suite if these pass cleanly.

149. assistant (2026-07-03T17:50:49.035Z)

That test used a temp repo outside the real checkout, so the resolver quite reasonably searched the actual repo root. I'm setting the supported `RALLY\_GOLDEN\_SEARCH\_ROOTS` env var in the test to model an out-of-tree corpus root.

150. assistant (2026-07-03T17:50:59.336Z)

The out-of-tree test is fixed to use the resolver's documented env root. I'm rerunning the focused pack once more.

151. assistant (2026-07-03T17:51:34.622Z)

Focused checks are clean. I'm rerunning the full CI-style suite again with the added `fusion\_audio` regression test, plus the stale-baseline nightly smoke.

152. assistant (2026-07-03T17:55:04.276Z)

Full tests, ruff, and mypy are green again; nightly still returns the expected stale-baseline status for A4 after scoring all 15. I'm reviewing the diff and then I'll amend the PR commit and push the update.

153. assistant (2026-07-03T17:55:14.092Z)

The review-response diff is scoped to the four comments. I'm doing the branch/HEAD safety check again, then I'll stage those six files and amend the PR commit.

154. assistant (2026-07-03T17:55:17.179Z)

Branch and commit stack are correct. I'm staging only the review-response files now.

155. assistant (2026-07-03T17:55:24.486Z)

The intended files are staged. I'm checking the staged set and then amending the existing A2 commit.

156. assistant (2026-07-03T17:55:36.389Z)

The commit is amended. I'm verifying the one-commit stack again, then I'll update the PR branch with `--force-with-lease`.

157. assistant (2026-07-03T17:55:40.020Z)

The branch is clean and still just one A2 commit over master. It's behind the old remote commit only because of the amend, so `--force-with-lease` is the right update.

158. assistant (2026-07-03T17:55:51.479Z)

The PR branch is updated. I'm refreshing the PR metadata and thread state now; then I'll update the PR body's validation numbers so it matches this head.

159. assistant (2026-07-03T17:56:00.682Z)

The thread fetch confirms the hardening line became outdated, while the skip-reason and constant threads still anchor to lines that changed but remain present. I'm updating the PR body's validation numbers and noting the added audio path coverage.

160. assistant (2026-07-03T17:56:23.084Z)

PR body is refreshed. I'm doing the final local/remote status check now; then I'll stop here as requested earlier.

161. assistant (2026-07-03T17:56:33.387Z)

Addressed the #452 review comments and pushed the updated PR branch.

Changes made:

- Wired `backend/eval/fusion\_audio.py` through `load\_resolved\_manifest`.
- Preserved resolver diagnostics in `tests/test\_served\_gate\_a\_regression.py` skip reasons.
- Hardened `golden\_manifest`:
 - ambiguous basename matches now produce manifest issues
 - trajectory fields no longer silently fall back to `output/`
 - resolver returns explicit rebase records
- Deduped `DEFAULT\_MANIFEST` so `rally\_seg\_eval` imports it from `golden\_manifest`.
- Added regression coverage for the `fusion\_audio` stale-manifest path.

Validation rerun:

- `ruff check .`
- `mypy backend training`
- `python run\_all\_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70`
 - `1608 passed, 6 skipped`, coverage `85.64%`
- Real n=15 manifest smoke still passes.
- `serve\_contrast` still runs n=15.
- Nightly regression still scores n=15 and exits stale-baseline as expected for A4.

PR body is updated. I did not resolve/reply to GitHub threads directly since you asked me to address comments, not manage thread state.

162. user (2026-07-03T18:07:47.406Z)

feel free to merge this PR and move ahead with the remaining items!

163. assistant (2026-07-03T18:08:04.281Z)

Great, I'll merge #452 first, sync `master`, then pick up the next non-conflicting alpha-readiness items. I'll keep avoiding A3 since that was already assigned elsewhere.

164. assistant (2026-07-03T18:08:09.251Z)

#452 is clean and all CI checks are green. I'm merging it with the repo's squash-style PR history, then I'll sync `master` before starting the next branch.

165. assistant (2026-07-03T18:08:25.961Z)

`gh pr merge` tripped over another local worktree that already has `master` checked out, so I'm switching to the GitHub API merge path. That avoids touching local branches and still performs

the same squash merge on GitHub.

166. assistant (2026-07-03T18:08:40.988Z)

#452 is merged on GitHub. I'll clean up the remote branch, fetch the new `origin/master`, and then branch from `origin/master` for the next item so we don't touch the busy local `master` worktree.

167. assistant (2026-07-03T18:08:50.716Z)

Remote cleanup reports the branch is already gone, and #452 is merged at `a68f02e`. I'm starting the next branch from `origin/master`; I'll begin with A4 because it is now unblocked by A2 and stays in this repo.

168. assistant (2026-07-03T18:08:54.314Z)

The worktree is clean; the stale tracking reference is harmless. I'm cutting the A4 branch from the remote base now.

169. assistant (2026-07-03T18:09:12.138Z)

For A4 I'm going to snapshot the n=15 nightly baseline on top of the newly merged manifest portability work, then immediately run the check mode to prove the stale warning is gone. I'll record the exact commands/results in the alpha tracker with A2 marked done and A4 marked as this PR's scope.

170. assistant (2026-07-03T18:09:16.121Z)

A4's baseline currently already covers 15 videos, but it is stale because the tracked config definitions changed. I'm regenerating it now, then I'll run check mode to confirm it passes instead of returning exit 4.

171. assistant (2026-07-03T18:09:29.867Z)

The baseline snapshot wrote cleanly. I'm running the non-update nightly check next; this should be the proof that the stale-baseline warning is gone for A4.

172. assistant (2026-07-03T18:09:45.038Z)

A4 is behaving correctly now: update mode wrote the baseline, and check mode passes with n=15 instead of reporting stale. I'm updating the tracker with the exact command record and then I'll package this as a PR.

173. assistant (2026-07-03T18:09:59.712Z)

The baseline diff shows exactly the intended refresh: default/milestone now match the current served config snapshot, while baseline-off remains the historical floor. I'm marking A2 as merged and adding an A4 refresh record with the pass-mode proof.

174. assistant (2026-07-03T18:10:23.347Z)

The tracker now has the command/output record A4 asked for. I'm running the lightweight validation for this docs/baseline PR first; because the only code-adjacent artifact is a baseline JSON, the nightly pass check is the main proof, but I'll also run the standard lint/type gates before committing.

175. assistant (2026-07-03T18:10:35.925Z)

The lightweight gates are clean. I'm going to pay the full-suite cost once for this baseline PR too, since the changed JSON is a regression artifact and we want CI to be boring.

176. assistant (2026-07-03T18:14:02.031Z)

Full suite is green. I'm doing the final diff check and then I'll commit/push an A4 PR.

177. assistant (2026-07-03T18:14:09.291Z)

The diff is just the refreshed baseline JSON plus tracker record. I'm re-verifying the branch and base before staging, then I'll commit explicitly.

178. assistant (2026-07-03T18:14:12.494Z)

Branch is correctly based on the merged A2 commit with no local commits yet. Staging only the A4 files.

179. assistant (2026-07-03T18:15:05.108Z)

I'm resuming from the staged A4 baseline refresh now. I'll re-check the branch and staged diff before committing, then push a small review PR and only continue into the next item if it's cleanly separable from that review gate.

180. assistant (2026-07-03T18:15:13.425Z)

I'm also reloading the GitHub publishing workflow instructions before the PR creation step, just to keep the repo discipline tight while this checkout may be shared with other sessions.

181. assistant (2026-07-03T18:15:27.218Z)

The staged scope is exactly the A4 baseline file plus the readiness tracker. I'm doing the final branch/HEAD/auth checks now, then I'll commit and push the branch.

182. assistant (2026-07-03T18:15:30.787Z)

Everything lines up: branch is on `origin/master`, HEAD is the merged A2 commit, and there are no extra commits riding along. I'm committing the A4 refresh now.

183. assistant (2026-07-03T18:15:38.009Z)

Commit is created as `60506b2`. I'm fetching once more and verifying that the branch contains only this A4 commit before pushing it up.

184. assistant (2026-07-03T18:15:42.962Z)

The branch is ahead by one commit, and that one commit is the A4 baseline refresh. I'll push it and open the review PR with the validation details.

185. assistant (2026-07-03T18:15:56.590Z)

The branch is on the remote now. I'm checking whether the GitHub connector can create the PR cleanly; if it can't, I'll use `gh pr create` with a real Markdown body.

186. assistant (2026-07-03T18:16:04.578Z)

Since A4 itself is reviewer-gated, I'm opening this one as ready for review rather than draft. The body will include the exact baseline command, the pass-mode proof, and the full local validation suite.

187. assistant (2026-07-03T18:16:44.698Z)

The connector hit the same private-repo 404 path, so I'm falling back to `gh pr create` now. Same title/body, ready for review.

188. assistant (2026-07-03T18:17:17.776Z)

A4 is up as PR #453. I'm checking its metadata/CI state, then I'll move to the next independent item from fresh `origin/master` so the review-gated baseline PR stays clean.

189. assistant (2026-07-03T18:17:27.113Z)

PR #453 is open and CI has started; the quick docs/lint checks are already green, with the heavier jobs still running. I'm going to take A5 next because it's independent of A3 and doesn't need the review-gated A4 baseline to merge first.

190. assistant (2026-07-03T18:17:30.933Z)

The A4 branch is clean. I'm cutting A5 directly from `origin/master` now, per the shared-checkout safety rules.

191. assistant (2026-07-03T18:17:36.297Z)

A5 branch is now based on remote master. I'm doing the required fast-forward pull on this branch before reading the tracker and evaluation scripts.

192. assistant (2026-07-03T18:17:46.588Z)

Now I'm tracing where the stale `heuristic\_lovo\_n6` floor is defined and how the n=15 heuristic run is produced, so A5 changes the right artifact instead of only changing wording in the tracker.

193. assistant (2026-07-03T18:17:55.336Z)

The tracker points at the Gen-0 harness floor file. I'm reading that harness and the existing `heuristic\_lovo\_n6` schema now to see whether the right A5 deliverable is a refreshed JSON artifact, a renamed n=15 floor, or a small compatibility update as well.

194. assistant (2026-07-03T18:18:06.749Z)

The floor file is just the heuristic per-video LOVO baseline used by the Gen-0 gate. I've found two plausible sources: the older `calibrate\_local` reproduction path and the newer manifest-resolved `rally\_seg\_eval`; I'm comparing them before choosing the least surprising refresh path.

195. assistant (2026-07-03T18:18:15.268Z)

I'm going to run the original `calibrate\_local` reproduction command against the now-portable n=15 manifest. If it produces the expected held-out per-video scores, I'll turn that into a refreshed floor artifact and docs record.

196. assistant (2026-07-03T18:20:26.069Z)

The n=15 calibration sweep is heavier than the old n=6 run and hit my two-minute timeout before printing. I'm rerunning with a longer cap; this is still CPU-only and should not touch the repo.

197. assistant (2026-07-03T18:35:43.594Z)

The original CLI is too slow on n=15 because it replays the same config/video predictions many times. I'm reading the calibration helper now and will compute the same LOVO result with cached per-video/per-config scores, then use that as the A5 evidence.

198. assistant (2026-07-03T18:35:49.122Z)

I also noticed two long-running Python processes still consuming CPU from the timed-out sweeps. I'm checking their command lines so I can stop only the orphaned calibration runs and leave unrelated user processes alone.

199. assistant (2026-07-03T18:35:54.714Z)

Those two CPU-heavy processes are exactly the timed-out `backend.eval.calibrate\_local` runs. I'm stopping only those PIDs before doing the cached computation.

200. assistant (2026-07-03T18:36:19.971Z)

For A5 I'm computing the exact same LOVO protocol but with prediction/score caching: 15 videos × 180 configs once, then the leave-one-video-out selection from the cached table. That should give us the n=15 floor without waiting hours.

201. assistant (2026-07-03T18:41:12.911Z)

The cached run finished: the n=15 calibrated heuristic floor is mean F1 `0.4456` with std `0.2390`, and every LOVO fold picked `(0.05, 1.0, 2.0, 0)`. I'm checking the existing tests before making this reproducible in code instead of leaving it as a one-off shell snippet.

202. assistant (2026-07-03T18:41:53.660Z)

I'm going to make `calibrate_local` cache its per-video/config score table and expose a floor JSON writer. That turns A5 from `?I ran a clever one-off?` into `?future Gen-0 comparisons have a repeatable n=15 baseline command?`

203. assistant (2026-07-03T18:42:38.885Z)

The calibration CLI now has a cached path and `--floor-out`. I'm adding a small unit test to pin that cached LOVO matches the generic LOVO semantics on a toy corpus, then I'll generate the n=15 floor artifact through the new CLI.

204. assistant (2026-07-03T18:42:50.524Z)

Now I'm running the new cached CLI to produce `eval_baselines/heuristic_lovo_n15.json`. It should take about the same four to five minutes as the proven cached computation.

205. user (2026-07-03T18:46:53.983Z)

the PR is merged

206. user (2026-07-03T18:46:54.003Z)

BTW this machine has GPU and you are free to use it

207. assistant (2026-07-03T18:47:10.242Z)

Got it, thanks. I'm going to fold the merged A4 state into the A5 branch carefully, and I'll use the GPU freely for the later feature-regeneration items; A5 itself is CPU-bound and already produced the n=15 heuristic floor.

208. assistant (2026-07-03T18:47:16.919Z)

Confirmed: A4 PR #453 is merged, and all CI checks were green. `origin/master` has advanced to the merge commit, so I'm doing the required fast-forward pull on the A5 branch now; the local A5 edits touch different files.

209. assistant (2026-07-03T18:47:32.758Z)

A4's merged changes are now fast-forwarded into the A5 branch. I'm updating the A5 docs/references so future Gen-0 runs point at the new n=15 floor while the old n=6 artifact remains historical evidence.

210. assistant (2026-07-03T18:47:43.654Z)

I'm keeping archived/historical n=6 references intact, but updating live runbook/examples and the alpha tracker. Next edit: mark A4 done, A5 in progress, add the exact A5 floor command/output, and point Gen-0 future examples at `heuristic_lovo_n15.json`.

211. assistant (2026-07-03T18:47:57.933Z)

I'm editing three small surfaces now: the alpha tracker, the Gen-0 harness example/help text, and `NEXT_STEPS` live guidance. The historical archive references stay as-is because they

describe the old n=6 experiment, not the new floor artifact.

212. assistant (2026-07-03T18:48:04.785Z)

Harness examples now point at the n=15 floor. Next I'm making the tracker carry the actual A5 evidence and PR state.

213. assistant (2026-07-03T18:48:22.111Z)

The tracker now has the A5 command and metrics. I'm updating the live 'what to run next' doc so future Gen-0 runs don't accidentally keep using the n=6 floor.

214. assistant (2026-07-03T18:48:41.464Z)

The live docs are updated. I'm checking the diff and running the focused tests/linters now; if those pass I'll run the broader suite before publishing A5.

215. assistant (2026-07-03T18:48:52.344Z)

The focused tests pass; ruff caught a small style issue in the new score helper. I'm tightening the type annotation to use the existing 'Score' object directly.

216. assistant (2026-07-03T18:49:20.160Z)

Mypy wants the helper dictionaries typed less narrowly, which is fair: these payloads are mixed JSON-shaped objects. I'm changing those return annotations to 'Any' for the dict values.

217. assistant (2026-07-03T18:49:31.380Z)

I'm re-running the focused gates after that typing cleanup.

218. assistant (2026-07-03T18:49:38.918Z)

One return type still says 'object', so mypy treats the per-video list as non-iterable. I'm fixing that last annotation.

219. assistant (2026-07-03T18:49:48.716Z)

Focused tests were already green; I'm rerunning ruff and mypy, then I'll do the full repo gates.

220. assistant (2026-07-03T18:50:11.402Z)

The focused gates are clean. I'm running the full local quality gate now: repo-wide ruff, repo mypy, the coverage-enforced full suite, and a Gen-0 harness smoke with the new n=15 floor.

221. assistant (2026-07-03T18:54:22.592Z)

Full gates are green, and the Gen-0 smoke consumed the new floor: learned mean stayed '0.551', 'floor_passed=True', sign test '9/15', p '0.3036', so it's still honestly not a ship claim. I'm doing the final whitespace/status pass before committing.

222. assistant (2026-07-03T18:54:39.728Z)

The working tree contains only the intended A5 files. I'm doing the branch/HEAD check required before staging so this commit doesn't accidentally ride on any sibling work.

223. assistant (2026-07-03T18:54:42.872Z)

HEAD is the A4 merge commit and there are no local commits ahead of 'origin/master'. I'm staging just the six A5 paths now.

224. assistant (2026-07-03T18:54:50.748Z)

The staging set is explicit. I'm giving the cached diff one last glance, then I'll commit.

225. assistant (2026-07-03T18:54:54.798Z)

The staged scope is exactly A5: cached floor generation, the n=15 floor JSON, tests, and live docs. I'm committing it now.

226. assistant (2026-07-03T18:55:02.506Z)

Commit `c27ef81` is created. I'm fetching and verifying the branch still contains only this A5 commit before pushing and opening the PR.

227. assistant (2026-07-03T18:55:08.077Z)

The branch is still a single clean commit on top of `origin/master`. I'm pushing A5 and opening the review PR.

228. assistant (2026-07-03T18:55:16.125Z)

The branch is pushed. The GitHub connector 404'd on this private repo earlier, so I'm using `gh pr create` directly for this one too.

229. assistant (2026-07-03T18:55:37.272Z)

A5 is up as PR #454. I'm checking CI state, then I'll move to the next independent item while that runs.

230. assistant (2026-07-03T18:55:48.177Z)

A5 PR #454 is open and CI is running. I'm taking A6 next because it's independent of A5 and A3: fix the `backend.eval.ablation` circular import, then run the n=15 ablation.

231. assistant (2026-07-03T18:55:54.991Z)

Using the debugging workflow for A6: reproduce the CLI failure, isolate the import loop, make the smallest fix, then pin it with a regression test and run the ablation command.

232. assistant (2026-07-03T18:55:59.456Z)

The A5 branch is clean. I'm cutting A6 directly from `origin/master` now so it doesn't depend on the open A5 PR.

233. assistant (2026-07-03T18:56:03.139Z)

A6 is on a fresh remote base. I'm doing the required fast-forward pull before reading and reproducing the ablation failure.

234. assistant (2026-07-03T18:56:09.632Z)

Now I'm reproducing the reported A6 import failure with the exact tracker command.

235. assistant (2026-07-03T18:56:21.788Z)

Reproduced: `ablation.py` imports `ablation\_pdf` at module bottom, while `ablation\_pdf` imports helpers back from `ablation.py`, so `python -m backend.eval.ablation` trips on partial initialization. I'm reading those modules to pick the smallest import-boundary fix.

236. assistant (2026-07-03T18:56:27.603Z)

The fix is small: keep `ablation.export\_pdf` as a public wrapper, but import the PDF module lazily inside the wrapper instead of at `ablation.py` import time. That preserves API compatibility and breaks the cycle.

237. assistant (2026-07-03T18:56:34.812Z)

I've applied the lazy import wrapper. I'm rerunning the exact failing CLI now to confirm the circular import is gone and see the n=15 ablation result.

238. assistant (2026-07-03T18:56:54.318Z)

The import fix worked, but the current ablation command still reports `n=6` because `ablation.load\_videos` deliberately skips the newer windowing-only feature JSONs. I'm checking the R2 metric docs now to see whether A6 should record that as an A7 dependency, or whether there's a separate n=15 R2 ablation command to run from the cached trajectories.

239. assistant (2026-07-03T18:57:04.698Z)

The R2 design says the R2 gate validation lives in `rally\_seg\_eval.compare()`, not the fusion-feature `ablation.py` table. I'm checking the windowing presets and then I'll run an n=15 R2/tIoU comparison that uses all cached trajectories.

240. assistant (2026-07-03T18:57:28.713Z)

I found the n=15 R2 comparison path: `rally\_seg\_eval --vs` over cached trajectories. I'm running baseline-off versus current served milestone so we get both `tol\_f1` and `f1@0.5` deltas across all 15 videos.

241. assistant (2026-07-03T18:57:46.733Z)

The n=15 R2 comparison ran cleanly: `tol\_f1` improved `+0.225` with CI excluding zero and 14/0 wins, while the net over-seg guard passed. I'm adding a regression test for the exact `python -m backend.eval.ablation --help` import path that failed, then I'll update the tracker.

242. assistant (2026-07-03T18:58:46.483Z)

I'm going to pick up exactly where the A6 branch left off: pin the import-cycle fix with a regression test, update the alpha readiness doc with the n=15 R2/tolF1 record, then run the focused checks before the full gate.

243. assistant (2026-07-03T18:58:56.114Z)

The branch is clean except for the intended `backend/eval/ablation.py` fix, and it's based on the merged A4 head. I'm reading the nearby tests and readiness tracker now so the doc/test changes land in the existing style.

244. assistant (2026-07-03T18:59:16.430Z)

One important bit shifted: your ?PR is merged? likely means `origin/master` may have moved under this A6 branch. I'm fetching remote now before I touch the tracker further, so this PR can sit on the real current head rather than fossilizing the previous doc state.

245. assistant (2026-07-03T18:59:32.517Z)

Remote `master` is still `d72a1c7`, so A5 has not landed here yet; A6 can stay based on the current remote head. I'm keeping the doc update scoped to A6's facts and leaving the A5 PR state alone.

246. assistant (2026-07-03T18:59:48.339Z)

I'm going to add one narrow regression test for the module entrypoint and update the readiness tracker with the measured A6 result: CLI import cycle fixed, R2/tolF1 n=15 contrast recorded, full fusion still explicitly blocked on A7.

247. assistant (2026-07-03T19:00:20.891Z)

The patch is in: the test exercises the exact `python -m backend.eval.ablation --help` path that

broke, and the tracker now separates ?R2/tolF1 proved useful on n=15? from ?fusion ablation still needs A7 feature regeneration.?

248. assistant (2026-07-03T19:00:35.613Z)

I'm starting with the tight verification loop: the ablation test file, lint for the touched Python, and mypy on the split modules. If that stays green, I'll rerun the two A6 eval commands and then the full repo gate.

249. assistant (2026-07-03T19:01:09.883Z)

The focused Python checks are green: `tests/test\_ablation.py` passed, touched-file ruff passed, and mypy found no issues in the ablation modules. Now I'm rerunning the two eval commands that the doc records, so the PR body can cite fresh local output instead of memory.

250. assistant (2026-07-03T19:01:25.025Z)

Both recorded A6 commands pass and the numbers match the tracker. I'm moving to the repo-wide gate now: full ruff, full mypy target, and the coverage-enforced test runner.

251. assistant (2026-07-03T19:05:17.527Z)

Full gate is green too: ruff, mypy, and the coverage-enforced suite all passed with 1609 passed / 6 skipped and 85.65% coverage. I'm doing the branch-safety checks now, then I'll commit only the four touched paths and open the A6 PR.

252. assistant (2026-07-03T19:05:21.608Z)

Branch-safety checks look right: no commits ahead of `origin/master` yet, and the worktree only has the intended A6 files. I'm staging those exact paths and committing the fix.

253. assistant (2026-07-03T19:05:29.201Z)

The staged diff is exactly the four A6 files. Commit is next, then I'll push and open the review PR.

254. assistant (2026-07-03T19:05:37.192Z)

Commit `a3be909` is on the A6 branch. I'm doing the final ?only my commit ahead of master? check, then pushing it up.

255. assistant (2026-07-03T19:05:48.273Z)

The branch is pushed. I'm going to use the GitHub publish flow for the PR creation step now; if the connector is unavailable for this private repo, I'll fall back to `gh` like the earlier PRs.

256. assistant (2026-07-03T19:06:12.268Z)

The GitHub PR-create tool isn't exposed in this session, so I'm using the `gh` fallback after checking the local CLI/auth state.

257. assistant (2026-07-03T19:06:18.358Z)

`gh` is authenticated for `Khelsutra/badminton-highlight-indexer` with default branch `master`. I'm creating the A6 PR now with the validation summary and the ?fusion still waits on A7? caveat up front.

258. assistant (2026-07-03T19:06:41.525Z)

PR #455 is open. Before I hand back, I'm refreshing the session handoff so the next window knows A6 is up for review and A7 is the GPU-enabled work now unlocked.

259. assistant (2026-07-03T19:06:58.461Z)

There isn't a repo-root handoff in this checkout, and the broad memory search hit the size of the local Codex project store. I'm narrowing it to the project-specific entries rather than scanning the whole tree.

260. assistant (2026-07-03T19:07:25.723Z)

No project auto-memory directory exists on this machine, so I'm following your handoff convention with a repo-root `SESSION\_HANDOFF.md`. It'll be a tiny resume map, not another tracker, and I'll push it as a second docs-only commit on the same PR.

261. assistant (2026-07-03T19:07:46.895Z)

The handoff is now a concise resume file with the PR, validation, and next quality items. I'm committing just that file to the A6 branch and pushing the PR update.

262. assistant (2026-07-03T19:07:49.856Z)

The branch still contains only the A6 commit plus the new untracked handoff file. Staging the handoff explicitly now.

263. assistant (2026-07-03T19:07:59.434Z)

Only `SESSION\_HANDOFF.md` is staged for this follow-up commit.

264. assistant (2026-07-03T19:08:05.616Z)

The handoff commit is `2a71edd`. I'm pushing it to update PR #455, then I'll make sure the PR metadata still looks clean.

265. assistant (2026-07-03T19:08:12.858Z)

PR #455 is updated with the handoff commit. I'm refreshing the PR body so the extra file is explicit instead of a surprise in the diff.

266. assistant (2026-07-03T19:08:33.063Z)

The PR body now mentions the handoff file too. I'm checking the final branch status and PR state so I can give you a clean handoff.

267. assistant (2026-07-03T19:09:08.632Z)

A6 is up as a ready PR, not draft: <https://github.com/Khelsutra/badminton-highlight-indexer/pull/455>. I'm going to keep moving into A7 on a fresh branch from `origin/master`, since you've cleared GPU use and A7 is independent of the A6 import fix until the later A8 decision reruns.

268. assistant (2026-07-03T19:09:20.214Z)

A5 just landed while we were finishing: `origin/master` moved to `c8eec7c`, and GitHub marks A6 as dirty against the new base. I'm going to rebase A6 onto the new remote head and resolve any tracker overlap before starting A7.

269. assistant (2026-07-03T19:09:23.373Z)

The worktree is clean and A6 has only its two commits, so this is the good case: a straight rebase onto the now-current `origin/master`.

270. assistant (2026-07-03T19:09:33.653Z)

As expected, the only rebase conflict is the readiness tracker: A5 updated the same rows and A6 updated them again. I'm resolving it by preserving A5 as DONE/#454 and keeping A6 as this PR's active row with the recorded R2 result.

271. user (2026-07-03T19:09:38.342Z)

Could you resolve the merge conflicts of [Khelsutra/badminton-highlight-indexer#455](https://github.com/Khelsutra/badminton-highlight-indexer/pull/455) ?

272. assistant (2026-07-03T19:09:51.244Z)

Yep, I'm already in the rebase conflict for #455. The conflict is only in ``docs/ALPHA_LAUNCH_READINESS.md``, where merged A5 and active A6 both touched the progress table and record section; I'm resolving it by keeping A5 as shipped (``#454``) and A6 as this PR.

273. assistant (2026-07-03T19:10:18.219Z)

The tracker conflict is resolved in-file now: both A5 and A6 records are present, with A5 marked DONE/#454 and A6 still IN PROGRESS/this PR. I'm scanning for leftover conflict markers before continuing the rebase.

274. assistant (2026-07-03T19:10:27.437Z)

No conflict markers remain. Git still shows the doc as unmerged only because the resolved version has not been marked staged yet, so I'm staging that file and continuing the rebase.

275. assistant (2026-07-03T19:10:40.619Z)

The rebase actually applied both commits and updated the branch, but Git returned a cleanup warning about removing ``.git/rebase-merge``. I'm checking the repository state carefully before pushing anything.

276. assistant (2026-07-03T19:10:45.776Z)

The branch has the rebased commits on top of ``c8eec7c``, but Git still thinks a rebase is active because that metadata directory is left behind. I'm asking Git to continue/finish once more, which should clear the stale rebase state without touching working files.

277. assistant (2026-07-03T19:10:57.536Z)

Git's second continue confirms the metadata is stale and incomplete (``head-name`` is missing). I'm verifying that ``.git/rebase-merge`` is an empty leftover under this repo, then I'll remove only that directory so Git no longer reports a phantom rebase.

278. assistant (2026-07-03T19:11:00.994Z)

The stale rebase metadata directory is empty and inside this repo's ``.git``. I'm removing just that empty directory, then I'll recheck status.

279. assistant (2026-07-03T19:11:18.800Z)

The local A6 branch is now clean on top of merged A5; it diverges from the old remote A6 branch because rebasing rewrote the two PR commits. Before force-with-lease pushing, I'm checking the handoff content because its old commit hash/A5 note is now stale.

280. assistant (2026-07-03T19:11:29.370Z)

The handoff still references the pre-rebase commit and treats A5 as uncertain. I'm updating that tiny bit now, then I'll run the verification pass against the rebased branch.

281. assistant (2026-07-03T19:11:49.733Z)

The handoff is current now. I'm rerunning the conflict-resolution verification: marker scan, full lint/type checks, and the coverage-enforced suite on the rebased branch.

282. assistant (2026-07-03T19:15:50.175Z)

The rebased branch passes the full local gate: no conflict markers, ruff clean, mypy clean, and the coverage suite is green with 1610 passed / 6 skipped at 85.52%. I'm amending the handoff-only commit with the updated A5/rebase note, then I'll force-with-lease push the rebased PR branch.

283. assistant (2026-07-03T19:15:52.951Z)

Only `SESSION\_HANDOFF.md` is modified after the checks. I'm staging that exact file and amending the latest docs commit.

284. assistant (2026-07-03T19:16:05.934Z)

The handoff amend is done. Final safety check now: current branch, clean status, and exactly the two A6 commits over `origin/master`, then I'll update the PR branch with `--force-with-lease`.

285. assistant (2026-07-03T19:16:19.636Z)

The rebased branch has been force-with-lease pushed. I'm checking #455 on GitHub now; if GitHub still shows checks in progress that's fine, but the merge conflict should be gone.

286. assistant (2026-07-03T19:16:25.486Z)

GitHub no longer reports the dirty conflict state; #455 is on base `c8eec7c`, with CI restarted and currently queued/in progress. I'm updating the PR body's validation numbers to the post-rebase run so review sees the fresh gate result.

287. assistant (2026-07-03T19:16:47.382Z)

The PR body is updated with the post-rebase validation. I'm doing one last PR/status check so I can tell you exactly where it stands.

288. assistant (2026-07-03T19:16:58.791Z)

Resolved #455's merge conflict and pushed the rebased branch with `--force-with-lease`.

PR: <https://github.com/Khelsutra/badminton-highlight-indexer/pull/455>

What changed:

- Rebased A6 onto `origin/master` after A5 merged as `#454` / `c8eec7c`.
- Resolved `docs/ALPHA\_LAUNCH\_READINESS.md` by keeping A5 as `DONE | #454` and A6 as `IN PROGRESS | this PR`.
- Preserved both the A5 heuristic floor record and A6 R2/tolF1 record.
- Updated `SESSION\_HANDOFF.md` so it no longer references stale pre-rebase state.

Validation after rebase:

- No conflict markers found.
- `ruff check .` passed.
- `python -m mypy backend training` passed.
- Full coverage suite passed: `1610 passed, 6 skipped`, coverage `85.52`.

GitHub now reports the PR as `MERGEABLE`; CI is still running on the fresh push.

289. user (2026-07-03T19:22:57.414Z)

```
<environment_context>
  <current_date>2026-07-04</current_date>
  <timezone>Asia/Calcutta</timezone>
  <filesystem><workspace_roots><root>C:\Users\avidu\Projects\khelsutra-
guru</root></workspace_roots><permission_profile type="disabled"><file_system
type="unrestricted" /></permission_profile></filesystem>
</environment_context>
```

290. user (2026-07-03T19:22:57.448Z)

merged the PR. Can you tell me what do you plan to complete next? A1 is being worked on by another agent

291. assistant (2026-07-03T19:23:12.014Z)

I'll quickly sync the indexer repo and read the current tracker/handoff before choosing the next lane, so I'm not planning from the pre-merge branch state.

292. assistant (2026-07-03T19:23:19.501Z)

The checkout is still on the now-merged A6 branch and clean. I'm moving it back to `master` and fast-forwarding from remote before reading the launch tracker.

293. assistant (2026-07-03T19:23:36.893Z)

`origin/master` fetched to `5e2ab8f`, which includes the merged A6 PR. I couldn't check out local `master` because another active worktree owns it for the A1 agent, so I'm reading the remote head directly via `origin/master` without disturbing that worktree.

294. assistant (2026-07-03T19:23:53.816Z)

I'd pick up **A7** next.

A1 is owned elsewhere, A3 is also already with another agent, and A8/A9 are logically blocked until A7 exists. So the highest-leverage next thing for me is:

A7: regenerate/upconvert full-fusion feature JSONs for the newer 9 golden videos

Plan:

1. Start a fresh A7 branch from `origin/master` without disturbing the A1 worktree.
2. Inventory the 15 golden videos and identify exactly which 9 are windowing-only.
3. Trace the existing feature generation path and prefer true regeneration on the local GPU over fake ?schema padding.?
4. Produce full-fusion feature JSONs with provenance for all 15, or document any clip that cannot be regenerated cleanly.
5. Run the feature loaders plus `fusion\_compare`/ablation smoke checks to confirm they now see `n=15`.
6. Open an A7 PR with artifacts/code/docs, and update the tracker/handoff to mark A6 as DONE/#455 and A7 as this PR.

I would **not** make promotion decisions in A7 beyond ?the data is now feature-complete.? A8 should be the separate decision PR where we rerun `fusion\_compare`, group ablation, and served contrast and then explicitly promote/reject person fusion, Gate-A served default, etc.

295. user (2026-07-03T19:32:23.543Z)

Please take over A3 and A7,8,9 as well.. The other agent is only working on A1

296. assistant (2026-07-03T19:32:37.541Z)

Got it. I'll take ownership of A3 plus A7/A8/A9, and leave A1 alone for the other agent. I'm going to sync the relevant repos first, then I'll split this into sensible PR-sized chunks rather than trying to jam corpus provenance, GPU feature regeneration, decision reruns, and Gen-0 shadow work into one giant diff.

297. assistant (2026-07-03T19:32:53.990Z)

I can see the relevant repos locally, including `sports-data-collector`. I'm pulling the current branch of each repo before reading anything substantive, per your repo-freshness rule.

298. assistant (2026-07-03T19:33:08.823Z)

The indexer pull could not fast-forward because the current local branch tracks the now-

merged/deleted A6 branch. Since the worktree is clean, I'll create fresh work branches directly from `origin/master` for new work and avoid relying on the stale local branch.

299. assistant (2026-07-03T19:33:16.295Z)

I'm syncing `sports-data-collector` next for A3, then I'll read the tracker and source-video docs from remote-fresh checkouts.

300. assistant (2026-07-03T19:33:31.547Z)

For the indexer side, I'm switching this checkout onto a fresh branch from `origin/master` now, so later A7/A8/A9 work starts from the merged A6 state and not from the deleted PR branch.

301. assistant (2026-07-03T19:33:45.194Z)

The indexer branch is now fresh from remote head and pulled cleanly. I'm going to inspect A3's collector/vault material and A7's feature-generation path side by side, then I'll decide which PR to open first.

302. assistant (2026-07-03T19:33:55.213Z)

I'm reading the tracker, golden corpus docs, collector source manifests, and the feature generator entrypoints now. The aim is to turn ?take over A3/A7-A9? into concrete PR boundaries with the least cross-repo tangle.

303. assistant (2026-07-03T19:34:06.984Z)

The tracker confirms the dependency chain: A7 is the blocker for A8, and A9 waits on A8. For A3, the collector currently has `data/golden\_source\_videos.json` for only the original six, while the indexer has 15 eval entries. I'm drilling into the exact schemas/tests and the local source-video availability now.

304. assistant (2026-07-03T19:34:25.712Z)

A3 looks very concrete: collector has a six-video `golden\_source\_videos.json` and a test hard-coding those six; indexer's manifest has all 15. I'm now locating the nine newer source/proxy files on `G:\F:` so A3 can become a durable 15-row provenance manifest rather than just a doc claim.

305. assistant (2026-07-03T19:34:41.116Z)

The newer labeled source/proxy files are on `G:\My Drive\Sharing\Annotation Videos`; nothing matching those newer names appeared under the F-drive GoPro folder. The collector DB itself only knows the original six owned golden rows, so A3 will likely be a manifest/test/doc PR rather than a DB migration unless we decide to register the proxy videos too.

306. assistant (2026-07-03T19:34:55.815Z)

I found seven of the newer annotation-source/proxy videos in the annotation request folder. The two Boxhill clips (`GX010137`, `GX010141`) appear under the Boxhill sharing/run folders rather than the annotation request folder, so I'm tracing their labels/provenance separately before I write A3.

307. assistant (2026-07-03T19:35:18.587Z)

The Boxhill logs show the true run inputs were `GX010137\_1080p.mp4` and `GX010141\_1080p.mp4`; the sharing folder only has derived highlight/Gemini reels. I'm searching local disks for those exact inputs before I decide whether A3 can fully close all 15 or needs an explicit missing-source status for those two.

308. assistant (2026-07-03T19:38:38.626Z)

Two Boxhill source inputs are not present as files in the synced local/Drive folders I searched,

but their run logs contain the full 32-char source hashes used by the indexer run. For A3, I'm going to make that explicit in the manifest instead of pretending the local file is present. First I'm cutting a collector branch from `origin/master`.

309. assistant (2026-07-03T19:38:43.873Z)

I'm creating the A3 branch in `sports-data-collector` from the remote, so the provenance manifest PR is isolated from the indexer quality branch.

310. assistant (2026-07-03T19:38:56.092Z)

A3 branch is fresh and pulled. I'm hashing the seven newer source/proxy files that are actually present, and I'll use the recorded run-log MD5s for the two Boxhill source inputs that are missing locally.

311. assistant (2026-07-03T19:53:32.565Z)

The seven present newer videos are hashed now. The two Boxhill source inputs remain missing as local files, but the run logs preserve their exact full-file hash as the `video\_id` emitted by the indexer run, so A3 can honestly cover all 15 with two entries marked ?hash from run log, source file not locally present.?

312. assistant (2026-07-03T19:53:40.731Z)

I'm going to update the collector manifest schema to v2 rather than stretching the old six-video shape. The new version will distinguish present local sources from ?hash captured from run log, source not present locally,? while still requiring every one of the 15 eval videos to have a 32-char MD5 record.

313. assistant (2026-07-03T19:54:39.025Z)

The manifest now covers all 15 corpus entries, with present-local entries hashed from files and the two missing Boxhill run inputs marked honestly with their run-log hash source. I'm updating the collector tests so this contract is enforced instead of hard-coding the old six-video world.

314. assistant (2026-07-03T19:55:06.489Z)

The tests now enforce three things: 15 expected eval IDs, unique 32-char hashes for every entry, and DB-path consistency for the original six that are actually registered in the collector DB. I'm running the focused collector test and lint next.

315. assistant (2026-07-03T19:55:26.793Z)

The focused test is green and the JSON validates. I'm running the collector-wide test/lint gate now; this PR only changes a manifest plus its contract test, but the repo is small enough to check cleanly.

316. assistant (2026-07-03T19:55:46.312Z)

Collector-wide validation is green: `ruff check .` and all tests pass (`295 passed, 2 skipped`). I'm reviewing the final diff now, then I'll commit and open the A3 PR.

317. assistant (2026-07-03T19:55:50.065Z)

The diff is tight. I noticed one leftover helper constant in the test that isn't needed, so I'm trimming that before committing.

318. assistant (2026-07-03T19:55:58.789Z)

That cleanup is done; I'm rerunning the small test/lint pair and then committing the A3 collector PR.

319. assistant (2026-07-03T19:56:06.379Z)

Focused checks are still green after the cleanup. The full collector suite is quick, so I'm rerunning it once after the final diff before commit.

320. assistant (2026-07-03T19:56:21.605Z)

Full validation is green again. I'm doing the branch-safety checks now, then staging only the manifest and test file.

321. assistant (2026-07-03T19:56:24.832Z)

The branch is cleanly based on `origin/master` with no commits ahead yet, and only the two intended A3 files are modified.

322. assistant (2026-07-03T19:56:34.688Z)

Only the intended manifest/test update is staged. Committing A3 now.

323. assistant (2026-07-03T19:56:42.367Z)

A3 is committed as `30d3379`. I'll push and open the collector PR, with the two Boxhill source-file caveats called out clearly.

324. assistant (2026-07-03T19:56:52.410Z)

The A3 branch is pushed. I'm creating the PR now with validation and the explicit note that 13 sources are present-local while 2 Boxhill hashes are preserved from run logs because the source files are not currently synced locally.

325. assistant (2026-07-03T19:57:11.282Z)

A3 is now up for review: <https://github.com/Khelsutra/sports-data-collector/pull/62>. I'm moving on to A7 in the indexer branch next: first I need to inspect the golden feature extractor and identify exactly why the newer nine JSONs are windowing-only.

326. assistant (2026-07-03T19:57:16.827Z)

For A7 I'll treat it as its own PR: make the full-fusion feature extractor run on all 15, produce/refresh the `output/\*\_golden\_features.json` artifacts as needed, and leave promotion decisions for A8.

327. assistant (2026-07-03T19:57:27.789Z)

I'm back in the indexer branch from `origin/master`. I'm checking status and then reading the extractor/loader code plus a representative old-vs-new feature JSON.

328. assistant (2026-07-03T19:57:34.146Z)

The A7 branch is showing one commit behind `origin/master`, so I'm fast-forwarding it before running any extraction or edits.

329. assistant (2026-07-03T19:57:42.106Z)

Fast-forward pulled the latest `origin/master` changes, which look like A1-adjacent startup/deploy work from the other agent. I'm keeping my A7 edits away from that area and now checking the actual feature schemas.

330. assistant (2026-07-03T19:57:59.143Z)

The schema check confirms the exact A7 blocker: all 15 feature files exist, but the newer nine are only windowing stats. The extractor can already produce full fusion schema, but for newer entries it needs a reliable source-video path; I'm checking the manifest resolver before deciding whether to add a source-manifest input or embed paths.

331. assistant (2026-07-03T19:58:15.212Z)

The extractor currently guesses `video\_path` from the labels path, which worked for the original six but not for the newer Drive/proxy layout. I'm checking whether these output artifacts are tracked; if they are, A7 can include the refreshed feature JSONs plus a small reproducibility improvement to feed source paths from the collector manifest.

332. assistant (2026-07-03T19:58:25.456Z)

The live `output/\*\_golden\_features.json` files are not tracked, so A7 cannot be a normal code PR that carries those artifacts unless we deliberately add a new tracked artifact location. I'm checking the ignore rules and the golden-fixture docs before choosing the least messy storage path.

333. assistant (2026-07-03T20:00:08.442Z)

I'm going to continue from the A7 branch and first make the extractor smarter about the 15-video corpus: the current gap is that the newer videos live on Drive/F-drive paths, while the fusion extractor only knows how to infer old local paths from label CSV names. After that I'll run what we can locally and call out any source-video gaps cleanly.

334. assistant (2026-07-03T20:00:21.180Z)

The indexer branch is current against its remote, and the top-level folder is just a container rather than a git repo. I'm going to refresh the collector checkout too, then re-read the handoff/tracker bits so the next edits line up with the merged state instead of stale notes.

335. assistant (2026-07-03T20:00:31.635Z)

I've confirmed the recent merged stack is on this branch now: #454, #455, and #456 are all in HEAD. Next I'm reading the handoff plus the alpha tracker sources so I can split A3/A7/A8/A9 cleanly and avoid stepping on A1.

336. assistant (2026-07-03T20:00:40.617Z)

The handoff still says A3 belonged elsewhere, but your latest instruction supersedes that, and the collector PR is already up for A3. The indexer tracker confirms A7 is the first blocker for A8/A9, so I'm focusing there before touching downstream decisions.

337. assistant (2026-07-03T20:01:33.512Z)

I've found the specific extractor weakness: it treats any existing `\*\_golden\_features.json` as complete, so it skips the newer windowing-only files, and it has no way to join the collector's source-video manifest for the Drive/F-drive video paths. I'm going to patch that narrowly: add source-manifest wiring, an `only missing/stale fusion schema` mode, and tests for both behaviors.

338. assistant (2026-07-03T20:02:59.037Z)

The code change is in place now. I'm adding regression coverage around the two failure modes that bit the corpus: stale windowing-only JSONs must be treated as incomplete, and the collector's source manifest must be able to supply the actual source/proxy MP4 paths.

339. assistant (2026-07-03T20:03:32.053Z)

Targeted tests are green, and Ruff only caught import ordering in `fusion\_audio.py`. I'm applying the formatter fix now, then I'll rerun the targeted checks and preflight the actual local corpus/video availability.

340. assistant (2026-07-03T20:03:58.854Z)

The A7 tooling now has focused tests passing. I'm moving to the actual machine preflight: confirm the manifest resolves to 15, confirm which source videos are locally present from the

collector manifest, and confirm whether CUDA is available before starting any long extraction.

341. assistant (2026-07-03T20:04:17.549Z)

The real preflight is good: the n=15 manifest resolves locally, CUDA is available on the RTX 2070, and the source manifest has 13 present-local videos. The two missing source videos are still `GX010137` and `GX010141`; I'm going to smoke-run the smallest stale video first before launching the seven-video extraction batch.

342. assistant (2026-07-03T20:04:36.548Z)

The smoke run reached the GPU extractor but failed before reading frames because the local Python environment is missing the ByteTrack package (`trackers`). That's a real setup gap for A7, so I'm checking the dependency declarations rather than papering over it.

343. assistant (2026-07-03T20:04:49.886Z)

The repo already has a single-sourced `requirements-trackers.txt` for exactly this dependency, pinned to an immutable commit. I'm installing that one dependency into the local environment, then I'll retry the smoke extraction.

344. assistant (2026-07-03T20:05:12.506Z)

`trackers` installed successfully from the repo's pinned SHA. I'm retrying `mahadevpura\_1` now; this should also prove whether the torchvision COCO weights are cached/downloadable and whether the extractor can write a full-fusion JSON on this machine.

345. user (2026-07-03T20:11:11.130Z)

the machine rebooted... can you resume?

346. assistant (2026-07-03T20:11:26.226Z)

Yep, I can resume. The aborted smoke run writes the feature JSON only after extraction finishes, so I'll first check the branch, package install, and whether that file stayed windowing-only, then restart from the smallest video.

347. assistant (2026-07-03T20:11:42.624Z)

Both repos are still up to date, the local code edits survived, and `trackers`, torch, and torchvision are installed. I'm checking the partial-output state and GPU health, then I'll restart the smoke extraction.

348. assistant (2026-07-03T20:12:40.096Z)

The telemetry shows the pre-reboot run was doing real GPU work and had reached about 24.5% of `mahadevpura\_1`; the feature file is still the old windowing-only one, so nothing corrupted. I'm adding one safety improvement before rerunning: source videos must exist before we write ?full? features, so the two known missing Boxhill sources fail loudly instead of producing all-zero person features.

349. assistant (2026-07-03T20:12:55.184Z)

I added the fail-fast source-video check and kept the default detector behavior unchanged, with explicit `--cpu`/`--detector-model` knobs for safer reruns if the laptop GPU keeps thermally throttling. I'm updating the small tests that use stub detectors so they provide a dummy video path under the new check.

350. assistant (2026-07-03T20:13:16.463Z)

Those tests now mirror the real contract: if a tool is about to inspect video pixels or audio, a concrete source file must be present. I'm rerunning the focused checks before the next extraction attempt.

351. assistant (2026-07-03T20:13:48.506Z)

The focused checks are green again. Since the last run pushed the laptop GPU into thermal throttling before the reboot, I'm checking whether we can lower the GPU power cap for a steadier local run; if Windows refuses that, I'll use a cooler smoke configuration first and keep the final extraction decisions explicit.

352. assistant (2026-07-03T20:14:00.218Z)

The GPU reports a configurable 80W cap with a low minimum, so I'm going to try a conservative 55W cap before rerunning. That should trade speed for not riding the thermal limit.

353. assistant (2026-07-03T20:14:09.052Z)

Windows won't let this session lower the GPU cap without elevated permissions. I'm going to do a non-destructive smoke run into `output\A7\_smoke` using a lighter detector and sparse sampling, just to prove the whole extraction path can complete after reboot without touching the real corpus artifacts.

354. assistant (2026-07-03T20:15:48.760Z)

The cooler smoke run completed and produced a full-fusion JSON in `output\A7\_smoke`, so the code path is healthy. Before I spend GPU time on final artifacts, I'm doing one more targeted search for the two missing Boxhill originals; if those are absent, A7 can only be completed for 13/15 locally unless we restore those source files.

355. assistant (2026-07-03T20:16:05.446Z)

Good catch from the fresh F-drive search: the two ?missing? Boxhill originals are actually present now on `F:\[Khelsutra] GoPro Hero Black 12\GoPro Recorded Videos\Badminton - Boxhill - 17 June\GX010137.MP4` and `GX010141.MP4`. That means A3 needs a quick correction too; I'm hashing those two files and will update the collector manifest so A7 can attach all 15 source paths.

356. assistant (2026-07-03T20:16:27.965Z)

The F-drive hashes are different from the run-log-derived hashes, which tells us those run logs were keyed to `\_1080p` derivatives while the now-connected F-drive has the actual GoPro originals. I'm updating the A3 manifest to record the present originals honestly, and the tests will stop expecting those two to be ?missing.?

357. assistant (2026-07-03T20:16:32.579Z)

Minor PowerShell syntax snag while slicing the JSON; I'm just reopening the relevant manifest section and patching it directly.

358. assistant (2026-07-03T20:16:37.497Z)

I'm pulling the exact ISO timestamps for those two F-drive originals so the manifest stays machine-verifiable rather than hand-wavy.

359. assistant (2026-07-03T20:16:44.185Z)

I'm editing only the collector manifest and its narrow test: `GX010137` and `GX010141` move from run-log-only placeholders to present F-drive originals, with their own full-file MD5s and byte counts.

360. assistant (2026-07-03T20:17:05.448Z)

The manifest now records all 15 source paths as present-local. I'm simplifying the test accordingly so it validates the current truth: every entry has a concrete local path, byte size, timestamp, and `Get-FileHash` MD5.

361. assistant (2026-07-03T20:17:14.294Z)

Tiny naming cleanup next: the two Boxhill IDs are no longer ?hash-only,? they just retain a note about the earlier derivative run-log hash.

362. assistant (2026-07-03T20:17:24.412Z)

A3's local truth is updated. I'm running the collector's narrow manifest checks now, then I'll use that corrected source manifest back in the indexer so A7 sees all 15 paths.

363. assistant (2026-07-03T20:17:31.698Z)

The collector checks pass. I also noticed the collector README still says the manifest records only the six owned MP4s, which is now stale after A3; I'm correcting that doc line before committing the A3 update.

364. assistant (2026-07-03T20:17:36.496Z)

I'm changing that README sentence from ?six owned MP4s? to the current 15-video source/proxy manifest, including the note that uploads remain owner-gated.

365. assistant (2026-07-03T20:17:48.348Z)

The collector A3 delta is now three explicit files: manifest, its test, and the README sentence. I'm doing the branch/HEAD safety checks before committing and pushing the PR update.

366. assistant (2026-07-03T20:17:52.483Z)

Branch safety looks good: the collector PR branch has only the prior A3 commit over `origin/master`, and the working tree contains only the three intended A3 files. I'm committing that correction and pushing it to the existing PR branch.

367. assistant (2026-07-03T20:18:26.220Z)

A3 is pushed to the existing collector PR with the F-drive originals recorded. I'm going back to the indexer now; before launching long extraction, I'm checking the telemetry helper because I'd like the GPU run to stop cleanly on high temperature instead of risking another reboot.

368. assistant (2026-07-03T20:18:39.118Z)

The source manifest now resolves 15/15 from the indexer. I'm adding a `--max-gpu-temp-c` abort guard to `fusion\_golden`; it uses the existing telemetry snapshots, so long runs can stop with a Python error instead of letting the OS be the first thing to complain.

369. assistant (2026-07-03T20:18:51.300Z)

The guard is wired into the progress callback. I'm adding one small unit test with a fake telemetry sample so that the temperature abort remains pinned.

370. user (2026-07-03T20:19:00.093Z)

are you going to run another long gpu run? I have put the fans on max perf

371. assistant (2026-07-03T20:19:13.286Z)

Yes, I think it is worth doing now, with two guardrails: I'll keep the final extraction on the default full-fidelity settings (`frame\_skip=3`, default Faster R-CNN) so A8 isn't mixing schemas, and I'll run it under the new temperature abort guard so the process exits cleanly if the GPU climbs too high. Since you put fans on max perf, I'll start with the smallest stale video again and watch telemetry before launching the full remaining batch.

372. assistant (2026-07-03T20:19:39.775Z)

The focused checks pass with the temperature guard included. I'm starting `mahadevpura\_1` as a background process with logs in `output\a7\_mahadevpura\_1\_default\*.log`, then I'll poll the telemetry instead of blocking blind.

373. assistant (2026-07-03T20:19:59.067Z)

The launcher returned without printing the PID, so I'm checking the Python process and the log files directly before assuming it started.

374. assistant (2026-07-03T20:20:03.098Z)

The process is running as PID `30232`. It has attached all 15 source paths from the corrected collector manifest and is currently in the person-detection pass for `mahadevpura\_1`.

375. assistant (2026-07-03T20:20:46.071Z)

The fans are doing their job: at 7.4% the GPU is at 68C with no throttle flag, much better than the pre-reboot run. I'm going to let this continue and keep polling the telemetry.

376. assistant (2026-07-03T20:21:38.766Z)

Still healthy: 17.2%, about 20 sampled frames/sec, GPU around 71-72C, no throttle. This is slower than the sparse smoke, of course, but stable enough to continue.

377. assistant (2026-07-03T20:22:46.696Z)

We've passed the point where the previous run died: 31.9%, GPU 74C, steady. I'll keep the single-video run going to completion, then verify the output schema before deciding how aggressively to batch the rest.

378. assistant (2026-07-03T20:24:25.002Z)

Halfway through and still cool: 51.5%, GPU around 72C. The guard has not tripped, and the output log is progressing normally.

379. assistant (2026-07-03T20:26:02.704Z)

Now 71.1% complete, temperature still 72-73C. There was one transient `throttled` flag from a clock dip, not heat; the temperature guard is quiet.

380. assistant (2026-07-03T20:27:25.920Z)

Almost there: 88.2%, still only 72C. This looks stable enough that after it finishes I'll run the rest as a resumable batch with `--missing-fusion-only`, likely one background process but watched the same way.

381. assistant (2026-07-03T20:29:14.958Z)

The full-fidelity `mahadevpura\_1` extraction completed in about 8.4 minutes, stayed around 72C, and rewrote the real `output\mahadevpura\_1\_golden\_features.json` with shuttle+person schema. I'm launching the resumable remaining batch now; it will skip all already-full files and process only the stale windowing-only ones.

382. assistant (2026-07-03T20:29:36.050Z)

Same quiet launcher behavior as before, so I'm checking the live Python process and the batch log directly.

383. assistant (2026-07-03T20:29:40.395Z)

The remaining batch is running as PID `30148`; it skipped the three smallest full-fusion files and started `GX020094` (99 windows, total frame span 22,938). Initial temp is 56C, so the batch is starting cool.

384. assistant (2026-07-03T20:31:24.012Z)

`GX020094` is at 21.2%, about 70C. It's doing more variable work because 99 candidate windows overlap the frame stream, but thermals still look fine.

385. assistant (2026-07-03T20:33:27.523Z)

`GX020094` is about halfway through now, 72C. No thermal pressure. I'll keep letting the batch run; the output is resumable if anything interrupts, and each finished video gets written immediately.

386. assistant (2026-07-03T20:36:07.914Z)

`GX020094` is past 90%; the existing file still shows windowing-only because the extractor writes at video completion, so that is expected. Once it moves to the next video, I'll verify `GX020094` changed to full-fusion.

387. assistant (2026-07-03T20:36:18.414Z)

That poll command had a PowerShell pipe syntax error before it even slept; the batch process itself should still be running. I'm reissuing the status check with simpler formatting.

388. assistant (2026-07-03T20:37:50.666Z)

`GX020094` finished and is now full-fusion: 99 candidates, 30 positive windows, 40 GT rallies. The batch has moved on to `GX030094`; it's at about 10.9%, still around 72C.

389. assistant (2026-07-03T20:41:02.491Z)

`GX030094` is 43.5% through and holding 72-73C. One useful note: its regenerated candidate count will be 79 from the current extractor, not the old lean file's 30, which is expected because A7 is rebuilding from trajectories rather than ?upconverting? the old rows.

390. assistant (2026-07-03T20:45:14.958Z)

`GX030094` is at 73.7%, still cool at ~73C. This one is slower than `GX020094`, but stable; the old `GX030094` file remains windowing-only until this video reaches its final write.

391. assistant (2026-07-03T20:48:27.103Z)

`GX030094` is at 99.0%, still ~73C. It should move to the next stale video on the next poll.

392. assistant (2026-07-03T20:49:42.122Z)

`GX030094` finished cleanly: 79 candidates, 21 positives, 41 GT rallies. The batch is on `GX010141` now; this one is slower on the F-drive original but still cool at about 69-71C.

393. assistant (2026-07-03T20:54:54.624Z)

`GX010141` is slow but not hot: 28.9%, around 69-70C, ETA about 15 minutes for this video. The F-drive original is clearly much slower to decode than the G-drive proxies, but it is progressing steadily.

394. assistant (2026-07-03T21:01:06.727Z)

`GX010141` crossed halfway, still about 70C. It is decode-bound and steady at about 14 sampled frames/sec; roughly 11 minutes left for this one.

395. assistant (2026-07-03T21:08:18.232Z)

`GX010141` is 91.1% done now, around 68-70C. The ETA has dropped to just a couple of minutes; the file will flip to full-fusion when this video_done event lands.

396. assistant (2026-07-03T21:12:51.306Z)

Yes, one long GPU pass is already running, and the fans-on-max call was exactly the right move. It's the full-fidelity A7 extraction, not another smoke run; I'll keep it on the thermal guard and won't start any second GPU-heavy job in parallel.

397. assistant (2026-07-03T21:13:14.326Z)

It's currently on `GX010137`, about 16% through that video, with the GPU sitting around 68C and the 87C guard armed. I'm going to just babysit this pass and let it finish the remaining stale full-fusion files before touching audio or training work.

398. assistant (2026-07-03T21:14:41.526Z)

Still healthy: `GX010137` is now ~22% through, temps are steady at 67-68C, and stderr is empty. The run is using the default detector path, so this is the real A7 artifact pass rather than a shortcut.

399. assistant (2026-07-03T21:17:48.825Z)

`GX010137` has moved to ~41%, still under 68C with no errors. This one is the slow F-drive original; after it finishes the remaining videos should continue in the same resumable batch.

400. assistant (2026-07-03T21:21:56.650Z)

We're at ~60% on `GX010137`; it's warmed a little but only to 69C, still very comfortable against the guard. I'll keep the batch single-threaded so we don't compete with the detector pass.

401. assistant (2026-07-03T21:27:05.030Z)

Nice, `GX010137` is in the home stretch now at ~87%, and temps actually dipped back into the mid-60s. I'll wait for it to close out and then check which video the batch picks next.

402. assistant (2026-07-03T21:30:12.414Z)

`GX010137` completed cleanly: 56 windows, 31 positives, 34 GT rallies. The batch immediately moved to `Boxhill\_Doubles`; it's only a few percent in and running around 69-70C, still fine.

403. assistant (2026-07-03T21:35:20.782Z)

`Boxhill\_Doubles` is about 38% through and holding at 71C. That's still comfortably below the guard, so I'm letting it keep chewing.

404. assistant (2026-07-03T21:41:27.583Z)

`Boxhill\_Doubles` is past three quarters now, still 71-72C. The thermal guard hasn't needed to intervene, and the error log is still clean.

405. assistant (2026-07-03T21:45:36.615Z)

`Boxhill\_Doubles` finished cleanly: 59 windows, 12 positives, 43 GT rallies. The batch skipped `kushagra\_singles` because full-fusion already exists, and it's now on `mahadevpura\_2`, which is a longer scan but still steady at ~71C.

406. assistant (2026-07-03T21:52:45.856Z)

`mahadevpura\_2` is the current long pole: ~30% through, tracking around 71-72C and about 18 minutes estimated from the log. The batch is behaving well, so I'm leaving it alone.

407. assistant (2026-07-03T22:02:53.994Z)

`mahadevpura\_2` is through ~69%, still at 71C. The machine is looking stable enough that I'm

comfortable letting this finish rather than backing off mid-video.

408. assistant (2026-07-03T22:12:03.044Z)

`mahadevpura\_2` is done: 14 windows, 2 positives, 40 GT rallies. The batch has moved to `GX010128`, which is bigger again at ~45k frames, and the GPU is around 72-73C.

409. assistant (2026-07-03T22:22:11.132Z)

`GX010128` is at ~38%, still steady at 73C. This is another long one; after it lands I'll re-check whether only `Badminton\_BXH\_2` remains or if anything else is still stale.

410. assistant (2026-07-03T22:37:21.613Z)

`GX010128` is now ~87% through and still pinned at 73C. This one should close out in a few minutes; then I'll see what the batch has left.

411. assistant (2026-07-03T22:43:30.000Z)

`GX010128` finished: 51 windows, 15 positives, 52 GT rallies. Now it's on `Badminton\_BXH\_2` at ~7%; this looks like the last heavy full-fusion extraction in the current batch, with temps still ~73C.

412. assistant (2026-07-03T22:58:37.460Z)

`Badminton\_BXH\_2` is past halfway, ~55%, and the GPU remains stable at 73C. This does look like the final heavy video in this run, so we're on the last long stretch.

413. assistant (2026-07-03T23:14:02.220Z)

The GPU batch exited cleanly. `Badminton\_BXH\_2` finished at 29 windows, 1 positive, 44 GT rallies, and the last two entries were already full-fusion so they were skipped. I'm doing a schema/count check across all 15 now before moving to audio augmentation.

414. assistant (2026-07-03T23:14:06.178Z)

Tiny PowerShell wrinkle on my schema-check helper, nothing touched. I'm rerunning it with the Windows-friendly inline Python form.

415. assistant (2026-07-03T23:14:22.808Z)

Full-fusion schema is now present for all 15 videos. Audio is still present on only the original 6, so I'm starting the A8 audio augmentation pass next; this should be CPU/video-I/O work, not another GPU detector run.

416. assistant (2026-07-03T23:14:38.819Z)

The audio augmentation process was launched to log files; I'm checking whether it stayed up or failed fast before I assume it's actually underway.

417. assistant (2026-07-03T23:14:44.094Z)

Audio augmentation is running as PID `30592`; it attached all 15 source paths from the collector manifest and started with `adarsh\_avi\_singles`. So far stderr is empty.

418. assistant (2026-07-03T23:16:51.096Z)

Audio augmentation finished quickly and cleanly; it processed all 15 source videos. I'm verifying that every feature file now has audio blocks before running the metric comparisons.

419. assistant (2026-07-03T23:17:06.317Z)

The corpus is now 15/15 full-fusion and 15/15 audio-augmented. Next I'm running the A8 metric

passes in order so we can compare the refreshed n15 floor against the old assumptions.

420. assistant (2026-07-03T23:17:14.129Z)

First metric pass is mixed: person fusion only nudges F1 from 0.302 to 0.311 on n15, and the paired CI crosses zero. Running audio comparison next; that will tell us whether audio is useful enough for alpha gating or only as a shadow signal.

421. assistant (2026-07-03T23:17:21.994Z)

Audio improves the mean F1 to 0.329, but again the n15 paired CI crosses zero. Good signal, not a statistically clean alpha default yet. I'm running the grouped ablation now to locate the failure modes instead of treating the aggregate as the whole story.

422. assistant (2026-07-03T23:17:35.004Z)

Grouped ablation says audio and shuttle are suggestive, person is slightly negative inside noise, and `gate\_a` is structurally protected. I'm running the serve contrast next so the launch tracker has both detection and serving-side evidence.

423. assistant (2026-07-03T23:17:50.343Z)

Serve contrast is the clearest launch signal so far: Gate-A remains useful as a structural guard, but it regresses under served windowing and should not be promoted to default. I'm checking the segmentation eval command shape next so I can refresh that artifact against the same n15 manifest.

424. assistant (2026-07-03T23:17:56.664Z)

The segmentation CLI matches the prior tracker path. I'm running the served baseline versus served+R2/R4 style candidate and writing the JSON artifact under `output` for the local launch packet.

425. assistant (2026-07-03T23:18:14.673Z)

The segmentation refresh is strong: served+cap/inactivity improves F1 by +0.236 and passes the net over-seg guard on n15, though the strict per-video split-count rule still blocks. Before I inspect the A9 training code, I'm refreshing the checkout per your repo convention so I'm not reading stale files.

426. assistant (2026-07-03T23:18:29.611Z)

Fast-forwarded onto current `origin/master` cleanly; our four code/test edits are still the only tracked local changes. Now I'm locating the A9/TemporalMaxer pieces in the refreshed tree.

427. assistant (2026-07-03T23:18:41.557Z)

I found the existing Gen-0 harness and the docs that frame TemporalMaxer. The current code appears to have a one-command train/eval/gate flow already; I'm checking its CLI and floor expectations before I run the refreshed n15 shadow training.

428. assistant (2026-07-03T23:18:51.400Z)

A9 has an existing Gen-0 LogReg/TAL harness, not a TemporalMaxer implementation yet. I'm going to run that harness against the refreshed n15 floor as the required shadow-training baseline bundle first; then we can decide whether the actual TemporalMaxer swap needs a follow-on code PR or belongs in the same branch.

429. assistant (2026-07-03T23:20:57.456Z)

Gen-0 n15 shadow training is complete. It beats the refreshed heuristic mean (0.551 vs 0.446), but only 9/15 paired wins with $p=0.304$, so the gate stays non-serving. The bundle is local under `artifacts/rally\_tal\_gen0/0.1.1-n15-shadow.20260704/`; now I'm updating the tracker/handoff with

the concrete A7/A8/A9 results.

430. assistant (2026-07-03T23:21:22.653Z)

I'm going to keep the docs honest rather than triumphant: A7/A8/A9 produced useful local evidence, but the launch conclusions are mostly ?measurements are now valid; don?t promote defaults/model yet.? I'm patching the tracker and handoff with exactly that.

431. assistant (2026-07-03T23:21:48.613Z)

Tracker header and current-state bullets are updated. I'm patching the row statuses and adding the A7/A8/A9 command records next.

432. assistant (2026-07-03T23:22:06.932Z)

The tracker now reflects A6 merged, A3 in the collector PR, and A7-A9 as this branch?s in-progress review payload. I'm adding the measured command outputs so reviewers can reproduce the conclusions.

433. assistant (2026-07-03T23:22:44.185Z)

The tracker now has the concrete result blocks and updated DoD checkboxes. I'm refreshing `SESSION\_HANDOFF.md` so a restarted session won?t think we're still at A6/#455.

434. assistant (2026-07-03T23:22:47.324Z)

I removed the stale handoff and am writing the fresh one now; keeping it short and factual so it stays useful.

435. assistant (2026-07-03T23:23:12.475Z)

Docs are refreshed. I'm running the focused validation suite after the fast-forward to catch any drift from the new `origin/master` changes before I commit.

436. assistant (2026-07-03T23:23:27.260Z)

Focused validation is green: ruff on touched files, 23 targeted tests, and diff whitespace check. I'm going to run the broader local gates now because this branch is about measurement trust, and stale CI would be a very boring way to lose confidence.

437. assistant (2026-07-03T23:23:46.694Z)

Full `ruff` and `mypy backend training` are clean. I'm starting the full test suite with coverage now; this is the last bigger local gate before commit/push.

438. assistant (2026-07-03T23:27:12.137Z)

The full suite found the expected local-artifact fallout: the owner-box litmus tests still assume exactly six full-fusion JSONs, and now A7 has correctly made that 15. I'm updating those tests to accept the refreshed full corpus while preserving the old six-video litmus behavior for legacy artifact folders.

439. assistant (2026-07-03T23:27:22.324Z)

The failing litmus is doing its job: it spotted that its hard-coded ?6 real golden files? contract is now stale. I'm changing it into a dual litmus: preserve the legacy six-video expected values, and assert the refreshed fifteen-video Gate-A values when the full A7 corpus is present.

440. assistant (2026-07-03T23:27:55.415Z)

The litmus now records both known realities: old n6 and refreshed n15. I'm rerunning the ablation tests plus lint before trying the full suite again.

441. assistant (2026-07-03T23:28:22.901Z)

Ablation tests are green again. I'm rerunning the full coverage suite so the previous failure is fully cleared, not just locally explained.

442. assistant (2026-07-03T23:31:37.975Z)

Full suite is green now: 1627 passed, 3 skipped, coverage 85.44%. The first run failed only because the owner-box litmus was correctly stale for n15; the updated test now covers both n6 and n15. I'm checking the final diff and branch state before committing.

443. assistant (2026-07-03T23:31:52.195Z)

One small cleanup before commit: the source-manifest helper said 'override' in the CLI help but 'existing explicit path wins' in code, and '--fresh' should really mean fresh even when '--missing-fusion-only' is supplied. I'm tightening those two semantics now so the review surface is less self-contradictory.

444. assistant (2026-07-03T23:32:00.974Z)

I'm adding a tiny assertion for that override behavior so it doesn't drift again.

445. assistant (2026-07-03T23:32:11.435Z)

The override test is in. I'm rerunning the focused checks affected by this last cleanup, then I'll decide whether another full pass is worth the extra few minutes.

446. assistant (2026-07-03T23:32:41.592Z)

Focused checks are green after the cleanup. I'm going to spend the extra three minutes on one final full coverage run so the PR can say the full suite passed on the final tree, not 'almost-final tree'.

447. assistant (2026-07-03T23:35:59.102Z)

Final full suite passed on the final tree: 1627 passed, 3 skipped, coverage 85.44%. I'm doing a last 'mypy' and diff whitespace check, then I'll commit and open the PR.

448. assistant (2026-07-03T23:36:12.576Z)

Validation is clean. I'm following the shared-branch safety protocol now: fetch, verify the branch/HEAD, stage only the explicit files, then commit.

449. assistant (2026-07-03T23:36:20.212Z)

Fetch completed, branch is 'codex/a7-a9-quality-chain', and HEAD is still 'origin/master' before our commit. One PowerShell typo used 'origin\master'; I'm rerunning that check with the proper ref syntax before staging.

450. assistant (2026-07-03T23:36:23.472Z)

'origin/master..HEAD' is empty, so no sibling commits are riding along. I'm staging only the seven intended files now.

451. assistant (2026-07-03T23:36:28.358Z)

The index contains only the intended files. Committing the A7-A9 branch now.

452. assistant (2026-07-03T23:36:36.433Z)

Commit landed. I'm doing the post-commit safety checks now: branch status and 'origin/master..HEAD' should show only this commit.

453. assistant (2026-07-03T23:36:41.571Z)

Post-commit checks are clean: one commit ahead, and `origin/master..HEAD` lists only `42551b2`. I'm going to use the GitHub publishing workflow now to push and open the PR.

454. assistant (2026-07-03T23:36:54.315Z)

Using the GitHub publish skill: it defaults to a draft PR unless you explicitly ask otherwise, so I'll open this as draft/reviewable evidence rather than pretend the A9 TemporalMaxer question is closed.

455. assistant (2026-07-03T23:36:58.829Z)

GitHub CLI is present and authenticated, target is `Khelsutra/badminton-highlight-indexer` with base `master`. Pushing the branch now.

456. assistant (2026-07-03T23:37:12.262Z)

Branch is on GitHub. I'm opening the draft PR with the measured A7/A8/A9 results and the validation trail in the body.

457. assistant (2026-07-03T23:37:34.079Z)

Draft PR is open: #458. One last housekeeping bit: the handoff still says "open the PR" as future tense, so I'm updating it to point at #458 before I push the final branch state.

458. assistant (2026-07-03T23:37:46.066Z)

Handoff now points at #458. I'm amending that into the existing commit and force-pushing with lease so the PR stays as a single clean commit.

459. assistant (2026-07-03T23:37:48.951Z)

Only `SESSION\_HANDOFF.md` is modified after the PR creation, and we're still on the right branch. Amending now.

460. assistant (2026-07-03T23:37:53.572Z)

Amend complete; pushing with lease now and then I'll verify the PR points at the amended commit.

461. assistant (2026-07-03T23:38:04.108Z)

Push updated the PR branch. I'm doing the final status/PR verification now.

462. assistant (2026-07-03T23:38:20.887Z)

GPU run is done now. No long GPU job is still running, so the fans can come back down from max.

Opened draft PR: [#458 - [codex] refresh n15 fusion and shadow training](https://github.com/Khelsutra/badminton-highlight-indexer/pull/458)

What landed:

- A7: regenerated local full-fusion/audio substrate: `15/15` full-fusion, `15/15` audio.
- A8: recorded n15 decisions in [ALPHA_LAUNCH_READINESS.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/ALPHA_LAUNCH_READINESS.md):
 - person fusion: not promoted
 - audio: suggestive/shadow only
 - Gate A served default: rejected
 - R2/cap/inactivity remains the strongest quality result
- A9: ran Gen-0 n15 shadow training. It beats the heuristic mean, but is still non-serving: `9/15` wins, `p=0.3036`; #172 and WASB C3 still block. TemporalMaxer itself is not implemented in this branch, and I called that out explicitly.
- Updated [SESSION_HANDOFF.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-

indexer/SESSION_HANDOFF.md).

Validation on final tree:

- `ruff check .`
- `python -m mypy backend training`
- full suite: `1627 passed, 3 skipped`, coverage `85.44%`

Local generated artifacts stayed ignored/uncommitted under `output/` and `artifacts/`.

463. user (2026-07-04T00:44:18.399Z)

is the alpha launch tracker updated? What are the major items left in it?

464. assistant (2026-07-04T00:44:32.238Z)

I'll check the tracker from the synced branch so I don't answer from memory. Since the A7-A9 work is still in PR #458, I'll distinguish ?updated in the PR? from ?merged on master.?

465. assistant (2026-07-04T00:44:41.398Z)

The branch is up to date and #458 is still an open draft. I'm reading the tracker rows now and will summarize what remains versus what's now captured in the draft PR.

466. assistant (2026-07-04T00:44:54.206Z)

The tracker says A3 is still in collector PR #62, so I'm doing one quick GitHub check on that PR before I call it a remaining item.

467. assistant (2026-07-04T00:45:09.771Z)

Yes, the alpha launch tracker is updated in draft PR [#458](https://github.com/Khelsutra/badminton-highlight-indexer/pull/458), not merged to `master` yet. The updated doc is [ALPHA_LAUNCH_READINESS.md](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/ALPHA_LAUNCH_READINESS.md).

Major items left:

- **A1: Hosted alpha flow**: real page-driven endpoint with auth, upload -> process -> jobs -> compile -> download. This is the big product-alpha blocker.
- **A3: Collector/vault source manifest**: PR [sports-data-collector #62](https://github.com/Khelsutra/sports-data-collector/pull/62) is still open.
- **A7-A9: Review/merge #458**: quality evidence is recorded, but PR is draft. A9 has an open scope question: close with current Gen-0 non-serving bundle, or implement TemporalMaxer as a follow-up.
- **A10: Ship-gate target**: owner decision needed on metric, threshold, corpus slice, significance rule, and allowed claims.
- **A11: WASB C3 provenance / messaging**: resolve or explicitly defer with ?no commercial model sharing? copy guardrail.
- **A12: Corpus flywheel decision**: reopen or defer P10/P11 automation for tester corrections -> collector/train/reeval.
- **A13: Final alpha launch review**: runbook, consent/privacy copy, observability, rollback, known limits, owner GO/NO-GO.

The quality lane is mostly in good shape now: n15 portability, nightly/floor refresh, R2/tolF1, and fusion/person/Gate-A decisions are recorded. The main remaining launch risk is productization plus owner/legal launch decisions.

468. user (2026-07-04T00:49:59.973Z)

any review comments for [Khelsutra/badminton-highlight-indexer#457](https://github.com/Khelsutra/badminton-highlight-indexer/pull/457) ?

469. assistant (2026-07-04T00:50:13.107Z)

I'll check the PR directly rather than infer from the merge commit. I'll pull the repo first per your convention, then inspect review threads/comments on #457.

470. assistant (2026-07-04T00:50:22.894Z)

Using the GitHub review-comment workflow for thread-aware state, since flat comments can miss whether inline review threads are resolved or outdated.

471. assistant (2026-07-04T00:50:33.149Z)

The PR is already merged and has no top-level comments or review summaries from the basic view. I'm checking the review-thread API next, which is the more reliable answer for inline comments.

472. assistant (2026-07-04T00:50:42.993Z)

The REST review endpoints for #457 are empty. The helper defaulted to the current branch PR (#458), so I'm checking its arguments and then forcing it to #457.

473. assistant (2026-07-04T00:50:49.930Z)

Both REST review surfaces say no reviews/comments, and CI on #457 was green before merge. I'll still do one direct GraphQL review-thread query on #457 so the "no comments" answer is thread-aware too.

474. assistant (2026-07-04T00:50:58.711Z)

No, PR [#457](https://github.com/Khelsutra/badminton-highlight-indexer/pull/457) has **no review comments**.

I checked:

- top-level PR comments: none
- review submissions: none
- inline review threads via GraphQL: none

It was merged on `2026-07-03T21:06:37Z`, and its CI checks were green.

475. user (2026-07-04T00:51:15.131Z)

I mean do you think it was good or it missed something

476. assistant (2026-07-04T00:51:29.814Z)

I'll switch into a proper review stance for #457: look at the actual merged diff, not just whether GitHub had comments. I'll read the code-review skill and then inspect the changed files/behavior around the WASB batching path.

477. assistant (2026-07-04T00:51:46.474Z)

I've got the merged diff. Now I'm reading the changed WASB code around the new batching/prefetch path and the native runner config plumbing, because that's where a subtle miss would live.

478. assistant (2026-07-04T00:52:52.989Z)

I'm going to pin the one concrete "missed" item to exact lines so my answer is useful rather than just vibes.

479. assistant (2026-07-04T00:53:23.620Z)

Yes, I think #457 was good: scoped, parity-conscious, measured honestly, and CI was green across lint, mypy, full tests, golden fixtures, and smoke checks. It also did the right thing by keeping defaults unchanged and only enabling the faster L4 settings through the cloud-serving preset.

The main thing it missed is a small config guardrail: ``indexing.wasb.batch_size`` and ``prefetch_batches`` are accepted as plain optional ints in `[backend/config/models.py](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config/models.py:118)`, then passed through in `[native_wasb_runner.py](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/pipeline/detectors/native_wasb_runner.py:116)`. A negative value could either pass an invalid ``--batch-size`` or silently enable prefetch at depth 1 because ``_PrefetchIter`` clamps queue depth with ``max(1, int(depth))``. Not urgent, but I'd add a small follow-up PR to validate these as positive when set.

The other ?miss? is really strategic, not a flaw: #457 gives a measured ~30% speedup, but it does not solve the core GPU underutilization problem. It correctly leaves that as the next larger item: parallel feed / serving throughput work. So: good PR, no serious correctness concern, but I'd follow it with config validation and then move back to the bigger throughput path.

480. user (2026-07-04T00:53:24.455Z)

Can you take a look at review comments on #458

481. assistant (2026-07-04T00:53:38.399Z)

I'll inspect #458's review threads directly and sync the checkout first so I'm not reacting to stale context. I'm using the GitHub review-comments workflow for this one.

482. assistant (2026-07-04T00:53:48.058Z)

The local checkout is current. Next I'm fetching the thread-aware PR review state, since flat GitHub comments can miss whether a review thread is resolved or outdated.

483. assistant (2026-07-04T00:53:57.577Z)

#458 has one review with four findings, and the PR is still open/mergeable with CI green at that reviewed commit. I'm pulling the inline threads now so I can tell which ones are still active and where they land in the code.

484. assistant (2026-07-04T00:54:15.566Z)

I found four unresolved, non-outdated threads. I'm checking the surrounding code before I call them valid or overcautious; the first one is especially worth verifying because it's about a silent corpus-count gap.

485. assistant (2026-07-04T00:54:30.304Z)

The first three comments reproduce cleanly in the code: there's an actual predicate disagreement, the missing-source checks happen inside mutating loops, and the ``--missing-fusion-only`` branch currently changes only the log line. I'm checking tests now so I can separate ?must fix? from ?nice tightening.?

486. assistant (2026-07-04T00:54:37.223Z)

The test comment is also fair: the current litmus accepts ``{6, 15}``, which covers old/new corpora but would let the historical mixed-schema count of 11 slip by. That one is a cheap hardening fix, not a runtime bug.

487. assistant (2026-07-04T00:55:16.920Z)

I looked through #458's review state and the surrounding code. There are four unresolved, non-outdated review threads, and I think all four are valid. CI is green and the PR is mergeable, but I would not merge it as-is.

Priority:

1. ****Fix before merge:**** [backend/eval/fusion_golden.py](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/fusion_golden.py:42) treats `candidates: []` as full-fusion, while the canonical loader predicate in [backend/eval/experiment.py](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/experiment.py:185) says empty candidates are not fusion schema. That can create a silent corpus hole.
2. ****Fix before merge:**** [backend/eval/fusion_audio.py](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/fusion_audio.py:66) and [backend/eval/fusion_golden.py](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/fusion_golden.py:177) fail on missing source videos inside the processing loop, after earlier files may already have been mutated. Pre-validating missing source videos before writes is the right fix.
3. ****Fix or remove:**** `--missing-fusion-only` in [backend/eval/fusion_golden.py](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/fusion_golden.py:322) is currently a dead flag; it changes the log line but not behavior. I'd probably remove it unless we want to define stronger semantics.
4. ****Tighten test:**** [tests/test_ablation.py](C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/tests/test_ablation.py:551) accepting `{6, 15}` is too loose and could miss the old mixed-schema count regression. It should assert the loaded count matches the independent canonical full-fusion count, and still allow only the expected corpus sizes.

My read: the review was good and caught real issues, especially the first one. Recommended scope is to patch all four in one follow-up commit on #458.

488. user (2026-07-04T00:56:00.189Z)

oh so you want to merge PR458 and not incorporate the comments right now?

489. assistant (2026-07-04T00:56:11.819Z)

No, the opposite: I would not merge #458 as-is. The first two comments are real pre-merge fixes, and the other two are cheap cleanup, so I'm going to incorporate all four into the PR branch now.

490. assistant (2026-07-04T00:56:22.347Z)

I've got the change set in view. The main design choice is simple: make `fusion\_golden` defer to the same schema predicate the loaders use, then preflight missing source videos before any per-video writes happen.

491. assistant (2026-07-04T00:56:31.684Z)

I'm going to edit the implementation and tests now. The behavior change I'm choosing for `--missing-fusion-only` is the smallest meaningful one: when enabled, skip non-existing outputs too, so it truly processes only stale/non-full-fusion JSONs.

492. assistant (2026-07-04T00:56:46.756Z)

Small correction before I touch the flag: it's referenced in the launch tracker run command, so removing it would churn docs. I'm checking that doc text and will give the flag real behavior instead of deleting it.

493. assistant (2026-07-04T00:57:25.359Z)

I'm going to make the source-video preflight reusable from `fusion\_golden` so audio and golden extraction use the same error formatting. That keeps the behavior consistent without adding a new module.

494. assistant (2026-07-04T00:57:45.116Z)

One important knock-on: after the predicate fix, test fixtures that used `candidates: []` as

?already full? need to become real tiny fusion-schema fixtures. I'll update those so the tests match the loader contract instead of preserving the bug.

495. assistant (2026-07-04T00:58:07.772Z)

The schema predicate and extraction planning are patched. Next I'm moving the audio missing-video failure to a preflight so augmentation either starts cleanly or does not touch any feature JSONs.

496. assistant (2026-07-04T00:58:26.944Z)

I'm adjusting the tests that modeled ?already extracted? with an empty candidate list. Under the fixed contract, an already extracted fusion file needs at least one candidate carrying `shuttle` and `person`.

497. assistant (2026-07-04T00:58:33.856Z)

The preflight will now require derived or manifest source videos for any item that will actually be extracted, so I'm adding tiny placeholder `.MP4` files in the GPU-free run tests. That keeps the tests honest without invoking real video processing.

498. assistant (2026-07-04T00:59:03.498Z)

The patch missed some exact context because a few helper tests differ slightly. I'm applying it in smaller chunks so we don't drag unrelated lines along.

499. assistant (2026-07-04T00:59:06.460Z)

I found the affected fixture spots. I'll add a tiny fusion candidate helper and then swap only the run/resume fixtures that need to represent completed fusion output.

500. assistant (2026-07-04T00:59:16.454Z)

The helper is in. Now I'm updating the extractor tests in place: placeholder videos for preflight and full-schema candidates for resume/cache checks.

501. assistant (2026-07-04T00:59:27.741Z)

A couple of those blocks have slightly different spacing/context, so I'm narrowing the patch further. Tiny steps, less drama.

502. assistant (2026-07-04T01:00:28.066Z)

I'm adding three focused regressions now: empty candidate files are not ?full fusion,? source preflight aborts before any extraction, and `--missing-fusion-only` now only upgrades existing non-full JSONs.

503. assistant (2026-07-04T01:00:53.020Z)

The golden extractor side is covered. I'm adding the matching audio regression so a missing second source can't leave the first JSON audio-mutated.

504. assistant (2026-07-04T01:01:06.582Z)

The runtime fixes are now covered by focused tests. Last review point is the litmus count guard, so I'm tightening that to compare loader output against an independent full-fusion file count.

505. assistant (2026-07-04T01:01:28.134Z)

The code and tests are patched. I'm doing a quick diff scan now for style issues and to make sure I didn't accidentally alter unrelated behavior.

506. assistant (2026-07-04T01:01:36.179Z)

The diff looks scoped. I can already see one long comprehension that Ruff may dislike, so I'm going to run the focused tests and lint on just these files, then tidy anything it points at.

507. assistant (2026-07-04T01:02:09.827Z)

Focused lint is clean. One new audio test failed before reaching the preflight because the manifest fixture omitted required trajectory fields, so I'm fixing that fixture rather than weakening the production check.

508. assistant (2026-07-04T01:02:20.440Z)

That fixture is corrected. I'm rerunning the focused suite now; if it stays green I'll do the branch-safety check, commit only these files, and push the PR update.

509. assistant (2026-07-04T01:02:52.626Z)

Focused tests are green: 52 passed, and Ruff is clean on the touched files. I'm doing the required branch/commit sanity checks now so the follow-up commit only contains these review fixes.

510. assistant (2026-07-04T01:03:01.751Z)

I'm re-verifying the branch base before staging. This checkout is shared across sessions, so I'll only stage the five files I touched and confirm the PR range before pushing.

511. assistant (2026-07-04T01:03:07.451Z)

Branch range is clean: #458 currently has only its original commit over `origin/master`, and my working tree contains just the review-fix files. I'm staging explicit paths and committing now.

512. assistant (2026-07-04T01:03:15.891Z)

The review-fix commit is created. I'm doing one last range check, then I'll push it to the PR branch.

513. assistant (2026-07-04T01:03:18.432Z)

The PR branch is ahead by exactly the one review-fix commit. Pushing now.

514. assistant (2026-07-04T01:03:28.019Z)

The PR branch is updated on GitHub. Per the repo handoff convention, I'm checking whether the session handoff needs a small factual refresh for this new review-fix commit.

515. assistant (2026-07-04T01:03:35.040Z)

The handoff is stale now: it still says review comments are pending and lists tracked edits. I'm going to refresh those few lines and push that as a tiny second commit.

516. assistant (2026-07-04T01:03:51.221Z)

Handoff refreshed. I'm staging just that file, checking the branch range again, and pushing the small documentation checkpoint.

517. assistant (2026-07-04T01:03:54.252Z)

Only `SESSION\_HANDOFF.md` is modified now. I'm committing it separately so the code review fixes stay easy to inspect.

518. assistant (2026-07-04T01:04:04.590Z)

The handoff commit is pushed too. I'm doing a final status/PR check so I can give you a clean ?what changed / what passed / what's pending? summary.

519. assistant (2026-07-04T01:04:23.188Z)

No, I meant the opposite. I incorporated the #458 review comments and pushed the updates.

Pushed commits on `codex/a7-a9-quality-chain`:

- `6214f63` `eval: address n15 fusion review comments`
- `5de1ef2` `docs: refresh session handoff for pr 458`

Addressed:

- Reused the canonical fusion schema predicate so `candidates: []` is not treated as full-fusion.
- Added preflight source-video validation before `fusion\_golden` / `fusion\_audio` mutate outputs.
- Gave `--missing-fusion-only` real semantics: upgrade existing non-full JSONs only, skip absent/full outputs.
- Tightened the n15 litmus count guard so the old n=11 mixed-schema gap can't slip through.
- Refreshed `SESSION\_HANDOFF.md`.

Verification:

- `python -m pytest tests/test\_fusion\_compare.py tests/test\_audio\_features.py tests/test\_ablation.py -q` ? `52 passed`
- `python -m ruff check ...` on touched files ? clean

I left the GitHub review threads unresolved/comment-free for reviewer/owner verification, since you didn't explicitly ask me to resolve or reply on GitHub. PR #458 is still draft, and GitHub checks had not populated yet immediately after the push.

520. user (2026-07-04T01:04:23.680Z)

can you take a look at #62 comments as well

521. assistant (2026-07-04T01:04:39.973Z)

I'll check #62 with the same thread-aware review flow. Since that PR is likely in the collector repo, I'm going to locate/sync that checkout first before reading code.

522. assistant (2026-07-04T01:04:47.952Z)

Found the `sports-data-collector` checkout. I'm fast-forwarding it before reading anything, then I'll fetch PR #62's review threads.

523. assistant (2026-07-04T01:04:56.310Z)

#62 is open and mergeable with CI green. There's one top-level follow-up comment plus the review; I'm pulling inline thread state now so I can tell what's still unresolved.

524. assistant (2026-07-04T01:05:18.016Z)

There are three unresolved inline threads plus one top-level PR-body drift comment. I'm checking the manifest/test context now so I can separate "needs a code/data patch" from "just update PR description."

525. assistant (2026-07-04T01:05:46.579Z)

I agree with the main #62 review: it's not cosmetic. For the two Boxhill clips, the manifest currently joins A7 to 4K originals while the labels/trajectory are 1920-wide, so it can mis-scale person-cue features. I'm checking whether the referenced 1080p derivatives are actually present locally.

526. assistant (2026-07-04T01:05:51.987Z)

The PR body is indeed stale: it still says 13/15 present and Boxhill missing-run-log, while the manifest now says 15/15 present. I'm retrying the local file check with a cleaner PowerShell wrapper.

527. assistant (2026-07-04T01:06:11.195Z)

The F-drive originals are present, but the guessed G-drive derivative paths are not. I'm searching for the exact `GX010141\_1080p.mp4` / `GX010137\_1080p.mp4` filenames before recommending a manifest patch that may not be locally valid.

528. assistant (2026-07-04T01:06:28.516Z)

I did not find the 1080p derivatives on either F: or G:. That makes the top comment a little sharper: the manifest should not pretend the 4K original is the eval-consumed join target, but we also can't simply point `local\_path` at a local derivative that is not present. I'm checking the indexer manifest labels/paths for those two to see what fallback it would use.

529. assistant (2026-07-04T01:06:47.127Z)

The indexer manifest has only labels/trajectory and `frame\_width=1920`; it does not have a recoverable source path for those two. The G-drive folder has run logs and highlight outputs, but not the exact `\_1080p.mp4` source inputs. I'm checking the run logs for what was actually hashed.

530. assistant (2026-07-04T01:07:05.633Z)

The run logs confirm WASB processed `GX010141\_1080p.mp4` and `GX010137\_1080p.mp4` with the derivative MD5s in the comments, but those source derivative files are not present now. That means the honest collector fix is probably to record the eval source as run-log-only/missing locally, while preserving the F-drive 4K original as provenance rather than as the join target.

531. assistant (2026-07-04T01:07:18.629Z)

The earlier `30d3379` commit actually had the safer shape already: Boxhill eval source recorded as run-log hash-only, not a present 4K local path. I'm going to patch #62 back toward that truth, keep the found F-drive originals as provenance fields/notes, and add explicit proxy eval-source notes/tests. I won't resolve GitHub threads or edit the PR body unless you ask.

532. assistant (2026-07-04T01:07:34.334Z)

I'm editing the manifest so `md5`/`local\_path` mean ?eval join target,? not merely ?largest original we found.? For the two Boxhill clips, the eval join target stays the run-log 1080p hash; the F-drive original becomes explicit provenance.

533. assistant (2026-07-04T01:08:15.915Z)

Manifest patch is in. I'm tightening the test contract now: run-log-only entries are allowed only for those two Boxhill eval inputs, proxy entries must say they are eval-consumed, and notes are accessed safely.

534. assistant (2026-07-04T01:08:31.533Z)

The test now encodes the reviewer's expectation instead of widening silently. I'm also updating the README sentence so it doesn't imply every entry is a present local file.

535. assistant (2026-07-04T01:08:44.375Z)

Data and docs are patched. I'm validating JSON, running the focused test, and linting the touched files before committing.

536. assistant (2026-07-04T01:08:54.674Z)

Validation is green: JSON parses, focused tests pass, and Ruff is clean. I'm doing the branch-

safety check now and then I'll commit/push only these three files.

537. assistant (2026-07-04T01:09:01.417Z)

The diff now lines up with the comments. One small data-text cleanup remains: the manifest purpose should mention run-log eval-source hashes too, since two entries intentionally have no local path.

538. assistant (2026-07-04T01:09:11.925Z)

I'm rerunning the focused validation after that manifest text change, then I'll commit and push.

539. assistant (2026-07-04T01:09:18.102Z)

Focused validation is still green. I'm staging explicit paths and committing the #62 review fixes now.

540. assistant (2026-07-04T01:09:21.781Z)

Branch range is clean and the working tree has only the three intended files. I'm committing and pushing the PR update.

541. assistant (2026-07-04T01:09:38.323Z)

The #62 branch is pushed with the manifest/test fixes. I'm doing a final status/PR check, and I'll see whether the collector repo has a handoff file that needs a small refresh.

542. assistant (2026-07-04T01:09:48.161Z)

The PR is open, non-draft, and mergeable; CI has started on the new commit. I'm going to poll once or twice so I can tell you whether it finished cleanly.

543. assistant (2026-07-04T01:10:17.142Z)

CI finished green on #62. GitHub still shows the old review threads as unresolved/non-outdated because they remain anchored, but the pushed commit changes the underlying data/test contract to address them.

544. assistant (2026-07-04T01:10:25.818Z)

I looked at #62 and pushed a fix.

The main comment was valid: the two Boxhill entries were pointing the manifest at the F-drive 4K originals, but the eval/trajectory source was the 1080p run input. I changed those entries back to run-log hash-only eval inputs, with `local\_path: null`, the `\_1080p.mp4` MD5s, and explicit `original\_\*` provenance fields for the F-drive originals.

Also addressed the smaller comments:

- Added notes to the 5 `annotation\_proxy` entries confirming the proxy is the eval-consumed join target.
- Tightened tests so `boxhill\_run\_input` is actually used only by the two run-log entries.
- Switched note checks to safe `entry.get("notes", "")`.
- Updated README wording for present-local vs run-log-only entries.

Pushed commit:

- `9272f09` `data: align golden source manifest with eval inputs`

Verification:

- `python -m json.tool data\golden\_source\_videos.json`
- `python -m pytest tests\test\_golden\_source\_videos.py -q` ? `2 passed`

- ``python -m ruff check tests\test_golden_source_videos.py` ? clean`
- GitHub CI for #62 ? green

I did not resolve or reply to the GitHub threads, per the usual ?leave reviewer verification alone unless asked? rule.

545. user (2026-07-04T01:10:26.229Z)

please respond to the github comments as well always when responding!

546. assistant (2026-07-04T01:10:39.410Z)

Got it. I'll make that the working convention: when I address review feedback, I'll also reply on GitHub with what changed. I'm going back now to post replies on #62, and I'll also reply on #458 since I had intentionally left those quiet earlier.

547. assistant (2026-07-04T01:10:49.755Z)

For #62 I'll reply on each unresolved review thread, plus add a PR-level note for the top-level/body-drift comment. Then I'll do the same pass on #458's four threads.

548. user (2026-07-04T01:10:56.734Z)

you got a few more comments on #458 as well

549. assistant (2026-07-04T01:11:09.307Z)

Good catch. I've posted one #62 thread reply so far; I'm pausing that and switching to the fresh #458 comments now so I'm not working off yesterday's review state.

550. assistant (2026-07-04T01:11:21.008Z)

#458 has one new top-level re-review comment: all four review findings are accepted as fixed, but CI broke because ``tests/test_fusion_golden_telemetry.py`` lacks fake ``.MP4`` files for the new preflight. I'm patching that targeted helper now, then I'll reply on the PR after pushing.

551. assistant (2026-07-04T01:11:24.291Z)

The failing helper is exactly as reported. I'm adding the dummy video file in ``_write_manifest``, keeping the production preflight intact.

552. assistant (2026-07-04T01:11:40.110Z)

Patch is in. I'll run the telemetry tests plus the previously touched focused set, then a full local pytest if the quick pass is clean.

553. assistant (2026-07-04T01:12:10.160Z)

Focused tests are green now: 56 passed, and Ruff is clean for the touched file. I'm starting the full local pytest suite before I push, since the comment was specifically about the full-suite CI failure.

554. assistant (2026-07-04T01:14:42.044Z)

Full local pytest is green: 1631 passed, 3 skipped. I'm doing the branch sanity check, then I'll commit/push the one-line CI fix and reply on #458.

555. assistant (2026-07-04T01:14:47.008Z)

The PR branch contains only the intended #458 commits plus this one-line working-tree change. I'm staging that single file and pushing.

556. assistant (2026-07-04T01:15:00.238Z)

The CI fix is pushed as `ad458e1`. I'm posting GitHub replies now: one PR-level reply for the new CI comment, then thread replies for the original four comments so the discussion has the code-change trail.

557. assistant (2026-07-04T01:15:23.869Z)

The PR-level reply posted. My first attempt at batch thread replies tripped over PowerShell hashtable property syntax, so I'm rerunning those replies with explicit key access.

558. assistant (2026-07-04T01:15:39.275Z)

Still PowerShell being too clever with hashtables. I'm switching to four explicit calls; less fancy, more reliable.

559. assistant (2026-07-04T01:16:39.694Z)

GitHub accepted the PR-level reply, but the thread replies hit a transient `api.github.com` connection error. I'm retrying those explicit calls now.

560. assistant (2026-07-04T01:17:05.909Z)

The #458 GitHub replies are posted now: one PR-level reply for the new CI failure comment and four inline-thread replies for the earlier review findings. I'm checking CI on the new `ad458e1` push.

561. assistant (2026-07-04T01:17:12.792Z)

The new push has all checks green except the full-suite job, which is still running. I'll poll it, and then I'll return to the remaining #62 replies I paused.

562. assistant (2026-07-04T01:17:53.301Z)

While #458's full CI job runs, I'm finishing the paused #62 GitHub responses: two remaining inline replies, plus a PR-body update/comment so the stale 13/15 provenance note is no longer ambiguous.

563. assistant (2026-07-04T01:18:09.118Z)

#62's PR body is updated to reflect the current manifest truth. I'm posting the remaining thread replies and a short PR-level response.

564. assistant (2026-07-04T01:18:30.595Z)

#62 is now fully replied to as well: PR body updated, remaining inline threads replied, and a PR-level summary posted. I'm back to polling #458's full-suite job.

565. assistant (2026-07-04T01:18:44.921Z)

#458 is now green on GitHub as well: full suite passed and all other checks are success. I'm doing one last quick check for stale PR text, since the re-review called out the validation line.

566. assistant (2026-07-04T01:18:55.159Z)

The #458 body still had the old validation counts, so I'm updating the PR description too. No code change, just bringing the review surface in sync with what actually ran.

567. assistant (2026-07-04T01:19:16.452Z)

Both PR descriptions are now synced with the actual fixes and validation. I'm doing the final clean-tree/status check across both repos.

568. assistant (2026-07-04T01:19:25.374Z)

Absolutely, I'll treat that as the default going forward: when I address review feedback, I'll also respond on GitHub.

Done now:

- #458: fixed the new CI failure with `ad458e1`.
- #458: replied to the new PR-level comment and all four earlier review threads.
- #458: updated the PR description validation.
- #458: CI is now fully green, including full offline suite. PR remains draft and mergeable.

Also finished the paused #62 follow-through:

- #62: replied to the remaining inline threads.
- #62: posted a PR-level response.
- #62: updated the PR description.
- #62: CI is green and PR is mergeable.

Local #458 verification: `1631 passed, 3 skipped`; focused review set `56 passed`; Ruff clean on the touched file.

569. user (2026-07-04T01:19:25.750Z)

once all the comments are addressed and CI green. Pls merge both the PRs

570. assistant (2026-07-04T01:19:46.172Z)

I'll merge them, but first I'm going to re-check both PRs from GitHub and fast-forward both local checkouts so the merge decision is based on current CI/comment state, not cached state.

571. user (2026-07-04T01:19:48.617Z)

Also update the tracker accordingly

572. assistant (2026-07-04T01:20:01.645Z)

Got it. I'll update the alpha tracker before merging #458 so the merged branch carries the current PR/comment/CI state. I'm going to merge #62 first, then record that in the tracker on #458, let CI rerun, and merge #458 once it's green.

573. assistant (2026-07-04T01:20:17.338Z)

I'm going to mark the review threads resolved where we've replied with fixes, then merge #62. That leaves the tracker update free to reference the collector manifest as merged rather than ?pending.?

574. assistant (2026-07-04T01:20:35.383Z)

Before merging, I'm resolving the review threads we've answered with commits. Then I'll merge #62 and update the indexer tracker to reflect the final collector manifest state, including the two Boxhill run-log-only eval inputs.

575. assistant (2026-07-04T01:20:48.143Z)

All review threads are resolved. I'm merging #62 now with squash merge and branch cleanup.

576. assistant (2026-07-04T01:21:06.012Z)

#62 merge command completed. I'm verifying it, then I'll update `ALPHA\_LAUNCH\_READINESS.md` on #458 with the final collector-manifest reality before merging #458.

577. assistant (2026-07-04T01:21:20.018Z)

The tracker has an important stale line: it says the source manifest supplied all 15 paths

including F-drive Boxhill originals. That would reintroduce the coordinate-space problem, so I'm correcting it before #458 merges.

578. assistant (2026-07-04T01:21:56.613Z)

I'm updating the table to mark A3/A7/A8/A9 as done, and adding the Boxhill caveat so the tracker doesn't imply the 4K originals are valid eval inputs. That nuance matters for future fresh regeneration.

579. assistant (2026-07-04T01:22:35.475Z)

The tracker is corrected. I'm refreshing the handoff in the same commit so a future session starts from the merged #62 / merge-ready #458 state instead of the older review state.

580. assistant (2026-07-04T01:23:05.573Z)

Both tracker and handoff are updated. I'm doing a whitespace/range check, then I'll commit/push this docs-only refresh and wait for #458 CI to come back green again.

581. assistant (2026-07-04T01:23:20.634Z)

The docs diff is clean and scoped. I'm committing this tracker refresh and pushing it to #458 now.

582. assistant (2026-07-04T01:23:39.555Z)

Tracker update is pushed as `f49a110`. CI has restarted, so I'm waiting for the new run to go green before marking #458 ready and merging.

583. assistant (2026-07-04T01:23:48.734Z)

The tracker-refresh CI is running; the review threads are resolved. I'll keep polling until the full suite is green, then I'll update the PR body/comment, mark #458 ready, and merge it.

584. assistant (2026-07-04T01:24:35.999Z)

Most checks are green; full suite, Windows golden fixtures, and wheel smoke are still running. I'm waiting those out before merging.

585. assistant (2026-07-04T01:25:26.950Z)

Only the full offline suite is still running now; everything else has passed.

586. assistant (2026-07-04T01:26:18.926Z)

The full suite is taking the usual couple of minutes. I'm still holding the merge until that last check finishes green.

587. assistant (2026-07-04T01:27:14.918Z)

CI is fully green on the tracker-refresh head. I'm updating the PR body/comment with that final validation, then I'll mark #458 ready and merge it.

588. assistant (2026-07-04T01:27:35.783Z)

The PR body and comment are updated. I'm converting #458 from draft to ready and merging it now.

589. assistant (2026-07-04T01:27:51.743Z)

The ready command succeeded; I'm verifying the merge state now so I don't assume the second command completed silently.

590. assistant (2026-07-04T01:28:01.870Z)

Both PRs are merged. I'm syncing both local checkouts back to `master` so the workspace reflects remote head after the merges.

591. assistant (2026-07-04T01:28:29.345Z)

Local checkouts are back on `master` and fast-forwarded. I'm doing one final clean status check across both repos.

592. assistant (2026-07-04T01:28:49.638Z)

Done.

- Merged `sports-data-collector` #62: `c5b24bc`
- Updated the alpha tracker in #458 before merge
- Replied to and resolved the GitHub review threads
- Confirmed CI green on both PRs
- Converted #458 from draft to ready
- Merged `badminton-highlight-indexer` #458: `f4fe24f`

Both local checkouts are now on `master` and fast-forwarded to `origin/master`.

Tracker update included the important A3 correction: A3/A7/A8/A9 are marked complete, and the Boxhill 4K originals are documented as provenance-only while the actual eval inputs remain the run-log-only 1080p hashes.

593. user (2026-07-04T01:29:05.752Z)

well done@

594. assistant (2026-07-04T01:29:10.932Z)

Thanks. That was a satisfying little knot to untangle: PRs merged, tracker honest, and no 4K/1080p footgun left hiding in the launch docs.